Bahir Dar University

Bahir Dar Institute of Technology

Faculty of Computing

Requirement Analysis Document (RAD)

for

Industrial project on Bug Bounty Platform

Submitted to the faculty of computing in partial fulfillment of the requirements for the degree of Bachelor of Science in Cyber Security

Group members

Name	IDNumber
1. Niway Tadesse	1507460
2. Abraham Asimamaw	1504838
3. Melkamu Tesfa	1507064

Advisor : Yosef Worku

2025

Bahir Dar University, Bahir Dar Institute of Technology

Declaration

The Project is our own and has not been presented for a degree in any other university and all the sources of material used for the project have been duly acknowledged.

Name	Signiture
1.Abraham Asmamaw	
2.Niway Tadesse	
3.Melkamu Tesfa	

Faculty: Computing

Program: Cyber Security

Project Title: Bug Bounty Platform

This is to certify that I have read this project and that in my supervision and the students' performance, it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science

Advisor Name	**Signature**
1.Yosef Worku	.....

Examining committee members		**signature**	**Date**
1. Examiner1	.....	_____	-----------
2. Examiner2	.....	_____	-----------

It is approved that this project has been written in compliance with the formatting rules laid down by the faculty.

Roles and Responsibilities of the Group Members

List of Tasks	Melkamu	Niway	Abraham
Requirement Gathering & Analysis	✔	✔	✔
System Design (UML Diagrams)		✔	✔
Implementation & Prototyping			✔
Documentation & Final Report		✔	✔

Acknowledgment

First and foremost, we give thanks to Almighty God for granting us the strength, wisdom, patience, and good health required to successfully complete this project. Without His guidance and blessings, the accomplishment of this work would not have been possible.

We would like to express our heartfelt gratitude to our family members for their unconditional love, encouragement, moral support, and understanding throughout our academic journey. Their continuous motivation played a vital role in helping us overcome challenges during the development of this project.

Our sincere appreciation goes to our project advisor, Mr. Yosef Worku, for his invaluable guidance, constructive feedback, and continuous supervision throughout the project period. His professional advice, technical insights, and commitment greatly contributed to improving the quality and success of this work.

We would also like to extend our thanks to our friends and classmates for their cooperation, idea sharing, and encouragement. Their discussions and support were essential in refining our understanding and approach to the project.

Finally, we acknowledge all individuals and participants who directly or indirectly contributed to the success of this project, including instructors, technical staff, and anyone who provided support, advice, or resources during the course of this work.

List of Acronyms

Acronym	Full Form
RAD	Requirement Analysis Document
VDP	Vulnerability Disclosure Program
VRT	Vulnerability Rating Taxonomy
CRC	Class-Responsibility-Collaboration
UI	User Interface
FREQ	Functional Requirement
UC	Use Case
BR	Business Rule
NFR	Non-Functional Requirement
SSDLC	Secure Software Development Life Cycle
OWASP	Open Worldwide Application Security Project

List Of Figures

Image 1 . Use case Diagram 41
Image 2 . User Registration_State chart Diagram 52
Image 3 .vulnerablity report 52
Image 4 .program creation 53
Image 5 . activity diagram user registration 54
Image 6 .validate report 55
Image 7 . program creation_activity 55
Image 8 . Reward Approval 56
Image 9 .Dashbord 56
Image 10 .Sequence Diagram 58
Image 11 . Analysis Class Model 59
Image 12 . Class Diagram 71
Image 13 . Home page prototype 74
Image 14 .user registration prototype 75
Image 15 .Researcher Dashbord 76
Image 16 .less featured Analytics 77
Image 17 . Report Submission prototype 78
Image 18 .report submission 2 80
Image 19 . Bounty program Creation prototype 80
Image 20 . Customer pentest Dashboard 81
Image 21 . Customer Analytics Dashboard 82

List Of Tables

Table 1 . Functional Requirement 38
Table 2 .Bussiness Rule Documentation 47
Table 3 . Non Functional Requirement 64
Table 4 . Hardware Requirement 67
Table 5 . Software Requirement 68
Table 6 . CRC Analysis 69
Table 7 . Change Cases 72

Table of Contents

Declaration 7
Roles and Responsibilities of the Group Members *8*
Acknowledgment 8
List of Acronyms 9
Table of Contents 11
Abstract 14
Chapter One 15
Introduction 15
1.1 Background 15
1.1.1 Overview of Bug Bounty Platform 15
1.1.2 Background of the Organization 15
1.1.3 Organizational Structure 16
1.1.4 Mission, Vision, and Core Values 16
1.2 Statement of the problem 17
1.3 Objectives 18
1.3.1 General Objective 18
1.4 Methodology 19
1.4.1 Requirement Gathering Methods 19
I. Interviews with Local Ethical Hackers 19
II. Interviews with Organizations and Security Teams 20
III. Interviews with Cybersecurity Experts 20
IV. Questionnaires for Ethical Hackers 21
V. Document Analysis of Online Sources 21
VI. Literature Review 22

After gathering requirements, they were structured and modeled using an Object-Oriented (OO) approach. *22*
1.4.2 Analysis and Design Methodology *22*
Analysis and Design Tools *23*
1.4.3 Implementation Methodology *24*
Frontend Development Tools *24*
Backend Development Tools *24*
1.5 Feasibility 25
1.5.3 Time Feasibility *26*
1.6 Beneficiaries or Significance of the Project 26
1. Public and Private Organizations *26*
Key benefits include *27*
2. Ethical Hackers and Cybersecurity Professionals *27*
3. Platform Owners and Developers *27*
4. Government and Regulatory Bodies *27*
1.6.2 Significance of the Project *27*
1.6.3 Effects on Organizations and the Industry *28*
1.7 Limitations of the Project 28
1.8 Scope of the Project 29
1.9 Organization of the Project 32
Chapter Two 34
System features 34
2.1 The Existing System 34
2.1.1 How the Existing System Functions *34*
2.2 Proposed System 35
2.3 Requirement Analysis 38
2.3.1 Functional requirement *38*
2.3.2 System Use case *40*
2.3.3 Business Rule Documentation *46*
2.3.4 User Interface Prototype *50*
2.3.5 State chart diagram *52*
2.3.6 Activity Diagram *54*

2.3.7 Sequence Diagram *57*
2.3.8 Analysis Class Model *58*
2.3.9 Logic Model *59*
2.4 Non functional requirement 64
2.5.1 Hardware requirements *67*
2.5.2 Software requirements *68*
2.6 Key Abstraction with CRC Analysis 69
2.6.1 Conceptual Modeling: Class Diagram *71*
2.6.2 Identifying change cases *72*
2.6.3 User Interface Prototyping *73*
References 83
Appendeces 84
Sample Questionnaire Used During Requirement Gathering *84*
Interview Questions Used During Requirement Gathering *84*

Abstract

The increasing reliance on digital systems and online services has made cybersecurity a critical concern for organizations worldwide. Traditional security testing methods, such as periodic audits and penetration tests, are often insufficient to detect all vulnerabilities due to limited scope, time constraints, and cost. This project presents the design and development of a Bug Bounty Platform, a secure and structured environment that connects organizations with ethical hackers to identify and remediate software vulnerabilities efficiently.

The platform allows organizations to define program scope, manage vulnerability submissions, and reward valid security findings. Ethical hackers can safely report vulnerabilities, receive recognition, and earn monetary rewards, all within a legally protected framework. By leveraging crowdsourced security expertise, the platform enhances the security posture of participating organizations while fostering a collaborative cybersecurity community.

Key features of the platform include vulnerability reporting and tracking, triage and validation mechanisms, secure communication channels, reward management, and reputation systems for researchers. This project demonstrates how bug bounty platforms, such as HackerOne and Bugcrowd, can be adapted and implemented to improve proactive security measures, reduce the risk of cyberattacks, and promote ethical hacking practices.

Chapter One

Introduction

1.1 Background

1.1.1 Overview of Bug Bounty Platform

A bug bounty platform is a crowdsourced cybersecurity system that enables organizations to identify and fix security vulnerabilities by engaging ethical hackers (security researchers). Organizations publish the scope of their digital assets, while researchers responsibly discover, report, and validate vulnerabilities in exchange for monetary rewards. This approach provides continuous security testing and improves overall cyber resilience.

1.1.2 Background of the Organization

The proposed organization is a commercial crowdsourced bug bounty platform planned to be established within the next 1–2 years. The platform will be headquartered in Ethiopia and will primarily serve Ethiopian organizations while expanding to the African market.

Year of Establishment**:** Planned (2026–2027)

Location**:** Ethiopia

Number of Employees: It will have at least 10 employees

Operational Area: Ethiopia and Africa

1.1.3 Organizational Structure

The organization will adopt a structured management hierarchy to ensure efficiency and security:

- Chief Executive Officer (CEO): Founder of the company, responsible for overall strategic direction.
- Top Director / General Manager: Oversees daily operations and manages all departments.
- Platform Administrators: Control system configuration, user management, and access control.
- Security Specialists / Analysts: Validate and triage vulnerability reports submitted by hackers.
- Technical Staff: Maintain the platform infrastructure and security.
- Support Staff: Handle communication with clients and researchers.

1.1.4 Mission, Vision, and Core Values

Mission

To secure digital assets in Ethiopia and across Africa by providing a trusted, ethical, and efficient crowdsourced cybersecurity platform.

Vision

To become a successful and outstanding crowdsourced bug bounty company in Ethiopia and Africa.

Core Values

- Promoting ethical hacking and responsible disclosure.
- Strengthening cybersecurity awareness and skills.
- Creating a sustainable cybersecurity ecosystem.
- Trust, transparency, and fairness.
- Innovation and continuous improvement.

1.1.5 Basic Information about the Business Process

The platform automates the following business processes

- ➢ Organization onboarding and subscription management.
- ➢ Secure submission of asset scope by clients.
- ➢ Researcher registration and verification.
- ➢ Vulnerability discovery and report submission.
- ➢ Technical triage and validation of reports.
- ➢ Secure communication between stakeholders.
- ➢ Bounty approval, reward processing, and reporting.

1.2 Statement of the problem

Many organizations, especially smaller ones, struggle with identifying security vulnerabilities due to limited resources for internal testing or expensive external audits.

Existing vulnerability discovery approaches suffer from several limitations:

- √ High cost of traditional penetration testing
- √ Limited security expertise availability
- √ Lack of continuous security testing
- √ Poor coordination between organizations and ethical hackers
- √ Limited local platforms supporting regional cybersecurity communities

These challenges expose organizations to undetected vulnerabilities, data breaches, and financial loss.

1.3 Objectives

1.3.1 General Objective

To develop a secure, scalable, commercial crowdsourced bug bounty platform tailored for the Ethiopian context, facilitating responsible vulnerability discovery and disclosure to strengthen national and organizational cybersecurity.

1.3.2 Specific Objective

The specific objectives of this project are to:

- √ Enable Ethiopian organizations, both public and private, to launch, manage, and monitor customized bug bounty programs in an affordable and independent manner.
- √ Provide a trusted and centralized platform for local and regional ethical hackers to register, discover active bug bounty programs, submit vulnerability reports, and earn fair rewards.
- √ Implement an efficient and secure vulnerability reporting, triage, validation, and communication workflow that supports timely remediation of security issues.
- √ Integrate secure and reliable payout mechanisms that comply with Ethiopian financial systems and regulatory constraints.
- √ Build researcher reputation systems and leaderboards to recognize performance, encourage ethical behavior, and nurture Ethiopia's cybersecurity talent.
- √ Promote awareness and adoption of responsible vulnerability disclosure practices, contributing to a safer and more resilient digital ecosystem in Ethiopia.

1.4 Methodology

1.4.1 Requirement Gathering Methods

The requirement gathering process for the Crowdsourced Bug Bounty Platform was conducted using multiple complementary techniques to ensure accuracy, completeness, and relevance to the Ethiopian cybersecurity environment. Since Ethiopia has limited crowdsourced security platforms and few specialized security organizations, requirements were collected from local stakeholders, experts, and international references.

I. Interviews with Local Ethical Hackers

√ **Description**

Semi-structured interviews were conducted with local Ethiopian ethical hackers and cybersecurity students.

√ **Purpose**

- To understand challenges faced in accessing international bug bounty platforms
- To identify preferred vulnerability submission workflows
- To gather expectations on rewards, reputation systems, and legal protection

√ **Outcome**

The interviews helped define

- Hacker registration and verification requirements
- Bug submission formats and severity classification
- Reputation, ranking, and leaderboard features

II. Interviews with Organizations and Security Teams

√ **Description**

Interviews were conducted with local public and private organizations, including developers and IT/security staff.

√ **Purpose**

- To understand current security testing practices
- To identify gaps caused by lack of continuous security testing
- To gather requirements for organization dashboards and program control

√ **Outcome**

This method contributed to

- Program creation and scope definition features
- Triage workflows and communication tools
- Access control and reporting requirements

III. Interviews with Cybersecurity Experts

√ **Description**

Discussions were held with cybersecurity professionals and academic experts.

√ **Purpose**

- To validate responsible disclosure practices
- To identify legal and ethical requirements
- To ensure alignment with global security standards

√ **Outcome**

Helped define

- Disclosure timelines
- Bug validation processes
- Non-functional security requirements

IV. Questionnaires for Ethical Hackers

√ **Description**

Online questionnaires were distributed to ethical hackers to collect quantitative data.

√ **Purpose**

- To reach a wider hacking community
- To understand skill levels and tool preferences
- To gather feedback on UI and usability expectations

√ **Outcome**

Supported design decisions for

- User interface simplicity
- Reward and motivation mechanisms
- Platform accessibility

V. Document Analysis of Online Sources

√ **Description**

Extensive analysis was performed on international bug bounty platforms and online security programs, including Microsoft, Google, HackerOne, and Bugcrowd.

√ **Purpose**

- To study real-world vulnerability disclosure workflows
- To understand industry best practices
- To identify features required for commercialization

√ **Outcome**

Provided guidance on

- Program lifecycle management
- Bug triage and duplicate handling

- Severity-based reward models

VI. Literature Review

√ **Description**

A literature review was conducted using academic journals, standards, and guidelines.

√ **Sources included**

- OWASP Vulnerability Disclosure Guidelines
- ISO/IEC 29147
- Research papers on crowdsourced security

√ **Outcome**

Ensured the system follows:

- Ethical hacking principles
- Responsible disclosure standards
- Secure system design practice

Requirement Modeling Approach

After gathering requirements, they were structured and modeled using an Object-Oriented (OO) approach.

The Object-Oriented approach was selected over the structured approach because

- The system involves multiple interacting entities such as Hacker, Organization, Program, BugReport, and Reward
- OO modeling supports modularity, scalability, and maintainability
- It aligns with modern web application development and commercialization goals
- Structured modeling was considered less suitable due to the dynamic and evolving nature of crowdsourced security platforms.

1.4.2 Analysis and Design Methodology

The project adopts an Object-Oriented Analysis and Design (OOAD) methodology rather than a structured analysis approach.

The Object-Oriented methodology was selected because the Crowdsourced Bug Bounty Platform is composed of several interacting real-world entities such as Ethical Hacker, Organization and Staff. These entities possess attributes, behaviors, and relationships that are naturally and efficiently represented using object-oriented concepts.

❖ The OOAD approach provides the following advantages

√ **Modularity:** The system is divided into well-defined classes, making it easier to manage and understand.

√ **Scalability:** Supports future expansion such as mobile applications, automation, and regional commercialization.

√ **Maintainability:** System changes can be implemented with minimal impact on existing components.

√ **Reusability:** Common functionalities can be reused through inheritance and composition.

√ **Industry Relevance:** OOAD aligns with modern web-based and enterprise security system development practices.

The structured analysis approach was not selected because it focuses primarily on functional decomposition and process flows, which are less suitable for systems with complex interactions and evolving requirements such as crowdsourced security platforms.

Analysis and Design Tools

The following tools were used during system analysis and design

- **Lucidchart:** Used to create UML diagrams such as Use Case, Class, Sequence, Activity, and State Chart diagrams.
- **Draw.io:** Used for quick modeling, requirement discussions, and diagram refinement.

- **StarUML:** Used for detailed object-oriented modeling and diagram consistency.
- These tools enabled clear visualization of system behavior, structure, and interactions among components.

are selected to ensure responsive design, cross-browser compatibility, and accessibility in low-bandwidth environments.

1.4.3 Implementation Methodology

The implementation methodology defines the software development tools, technologies, and platforms used to develop, test, deploy, and maintain the system.

Frontend Development Tools

√ **Programming Languages:** HTML, CSS, JavaScript

√ **JavaScript Libraries:**

- jQuery (3.7.1)
- jQuery Migrate and jQuery UI
- core-js (3.47.0)

√ **UI Framework:** Bootstrap

These technologies

Backend Development Tools

√ Programming Languages: PHP and Python
√ Architecture: RESTful API
√ Web Server: Nginx
√ Caching: Varnish Cache
√ CMS: Wordpress
√ Security: HSTS,Google Recapcha
√ Blog: Wordpress

The backend architecture supports secure authentication, role-based access control, API-driven communication, and scalability.

Development Tools

√ **Version Control:** Git

√ **Development Environment:** Visual Studio Code

1.5 Feasibility

The feasibility of the Crowdsourced Bug Bounty Platform is analyzed in terms of economic feasibility, technical feasibility, and time feasibility to ensure that the project is practical, achievable, and sustainable within the given constraints.

1.5.1 Economic Feasibility

The project is economically feasible due to its reliance on open-source technologies and a crowdsourced security model, which significantly reduces development and operational costs.

The platform minimizes the need for expensive, periodic penetration testing by enabling continuous vulnerability discovery through ethical hackers.

Organizations benefit from affordable security testing services, which is especially important in the Ethiopian context where many institutions operate with limited cybersecurity budgets.

The system has strong commercialization potential through:

- Organization subscription fees
- Program listing fees
- Service commissions on rewards

Therefore, the system offers both cost efficiency for users and a viable revenue model for future commercialization.

1.5.2 Technical Feasibility

The project is technically feasible based on the existing technical skills and academic background of the project team and the availability of appropriate development tools and resources.

The development team consists of students with hands-on knowledge in web application development, database management, and basic cybersecurity principles, gained through university coursework, and self-learning. The team has practical experience on the mentioned technology stack,which are the core technologies required to implement the proposed system.

1.5.3 Time Feasibility

The project is time-feasible within the academic schedule of the university.The system will be developed based on the available time and divided into the SDLC phases such as requirement gathering, analysis,design, development,testing and deployment.

Advanced commercial features can be implemented in later iterations after the completion of the academic project.The use of agile and incremental development allows progress to be monitored and adjusted according to available time.

1.6 Beneficiaries or Significance of the Project

1.6.1 Beneficiaries

The deployment of the Crowdsourced Bug Bounty Platform provides significant benefits to a wide range of users and stakeholders. The system contributes to improving cybersecurity practices, strengthening responsible disclosure culture, and supporting digital transformation, particularly in the Ethiopian context where local bug bounty platforms and specialized security organizations are limited.

1. Public and Private Organizations

Government institutions, financial organizations, telecom companies, startups, and private enterprises directly benefit from the system by gaining access to an affordable and continuous vulnerability discovery mechanism. Instead of relying solely on expensive penetration testing services, organizations can launch customized bug bounty programs to identify security weaknesses early.

√ **Key benefits include**

- Improved protection of digital assets and sensitive data
- Reduced cybersecurity incidents and financial losses
- Faster vulnerability identification and remediation
- Increased trust from customers and users

2. Ethical Hackers and Cybersecurity Professionals

Ethical hackers, cybersecurity students, and independent security researchers benefit by having a legal and trusted platform to practice their skills and earn rewards.

√ **Benefits include**

- Legitimate opportunities to report vulnerabilities
- Fair recognition through reputation and leaderboard systems
- Skill development and career growth
- Encouragement of responsible disclosure practices

3. Platform Owners and Developers

The organization managing the platform benefits from commercialization opportunities through subscriptions, service fees, and partnerships. The platform can grow into a sustainable cybersecurity service provider within Ethiopia and the region.

4. Government and Regulatory Bodies

Government agencies benefit indirectly through

- Improved national cybersecurity posture
- Support for digital government initiatives
- Promotion of ethical hacking and responsible disclosure

1.6.2 Significance of the Project

The significance of his project lies in addressing a critical gap in Ethiopia's cybersecurity ecosystem.

This project

- Promotes a local and culturally relevant cybersecurity solution
- Reduces dependency on foreign security platforms
- Encourages ethical hacking and talent development
- Supports national digital transformation efforts

1.6.3 Effects on Organizations and the Industry

The platform positively impacts organizations by

- Lowering the cost of cybersecurity testing
- Improving compliance with security best practices
- Creating a collaborative security culture

For the broader industry, it contributes to

- Strengthening the local cybersecurity market
- Creating employment and skill development opportunities
- Enhancing innovation in security services

1.7 Limitations of the Project

The proposed Crowdsourced Bug Bounty Platform addresses key challenges in Ethiopia's cybersecurity landscape by providing a structured, localized mechanism for vulnerability discovery and responsible disclosure. However, due to time constraints, resource limitations, and the academic nature of the project, the system has certain boundaries and limitations.

The following challenges and constraints have hindered the full achievement of the planned objectives

- ❖ Limited Development Time: As a final-year undergraduate project with a fixed academic timeline, only the core functionalities could be analyzed and modeled in detail. Full implementation, testing, and deployment of a production-grade system were may not possible.
- ❖ Resource Constraints: Lack of access to commercial-grade infrastructure (e.g., high-availability servers, advanced security auditing tools) and budget for third-party integrations e.g real payment gateways.

- ❖ Scope Prioritization: Focus was placed on requirement analysis and design (as required by the RAD), rather than deployment and post-launch maintenance.
- ❖ Regulatory and Compliance Challenges: Detailed handling of Ethiopian financial regulations, tax reporting for bounty payouts, could not be fully incorporated due to time limits.

√ Activities left undone due to these constraints

- Full implementation and coding of the system.
- Integration with real payment gateways (e.g., Ethiopian banks, Telebirr, CBE Birr).
- Mobile application development (native iOS/Android apps).
- Advanced AI-based report triaging or researcher matching.
- Complete legal compliance module .

1.8 Scope of the Project

√ The scope of the Crowdsourced Bug Bounty Platform is defined to closely align with system requirements and bussiness logic, focusing on managed bug bounty programs while incorporating elements of vulnerability disclosure and researcher engagement.

√ The system boundaries include the following business processes and subsystems.

I. User and Role Management Subsystem

➢ Support for multiple roles: Organizations (customers), Ethical Hackers/Researchers (the Crowd), and Administrators/Platform Managers and staff.

➢ Registration, profile management, authentication, and researcher vetting/profile enrichment.

II. Program Creation and Management Subsystem

- Organizations can launch and manage public, private (invite-only), or managed bug bounty programs.
- Define scope (in-scope/out-of-scope assets), rules, reward structures (tiered bounties based on severity), and program briefs.
- Support for Vulnerability Disclosure Programs (VDP) as non-rewarded intake channels.

III. Researcher Engagement and Matching Subsystem

- Researchers can browse, join, and participate in programs.
- Basic matching of researchers to programs based on skills and performance (conceptual, without full AI implementation).

IV. Vulnerability Submission and Reporting Subsystem

- Researchers submit detailed reports with evidence, reproduction steps, impact assessment, and attachments.
- Structured submission workflows compliant with Vulnerability Rating Taxonomy (VRT)-like severity scoring.

V. Managed Triage and Validation Subsystem

- Platform-managed triage: Intake, validation, duplication checking, prioritization, and remediation advice.
- Status tracking (New → Triaged → To Review → Resolved) with notifications.

VI. Collaboration and Communication Subsystem

- Secure in-platform messaging and comments between researchers, organizations, and platform triage team.
- Real-time updates and activity feeds.

VII. Reward and Payout Management Subsystem

- Bounty awarding based on validity and severity.
- Payout tracking and researcher reputation/leaderboard system.

VIII. Analytics, Reporting, and Dashboard Subsystem

- Role-based dashboards for organizations (program insights, KPIs, vulnerability trends) and researchers (submissions, earnings).
- Basic reporting and benchmarking.

IX. Administration and Integration Subsystem

- Admin tools for program moderation, user management, and platform oversight.
- Conceptual support for API integrations and workflow connectors (e.g., to SDLC tools).

√ **Out of Scope** (will be included in the future)

- Full Penetration Testing as a Service (PTaaS) or Continuous Red Teaming.
- Attack Surface Management (automated asset discovery and risk ranking).
- Advanced AI features (e.g., CrowdMatch™, GenAI triage, Security Knowledge Graph).
- Managed Vulnerability Disclosure at scale without bounties.
- Mobile applications or external researcher credential verification.

1.9 Organization of the Project

This Requirement Analysis Document (RAD) for the Crowdsourced Bug Bounty Platform is structured in accordance with the guidelines provided by the Faculty of Computing, Bahir Dar Institute of Technology. The document consists of preliminary sections, two main chapters, references, and appendices, presenting a thorough requirement analysis and design foundation for the proposed commercial platform.

The preliminary sections comprise the title page, declaration (with signatures of group members, advisor, and examiners), roles and responsibilities of the group members (including the completed role assignment matrix), acknowledgment, list of acronyms, list of figures, list of tables, abstract, and table of contents. These sections provide formal documentation of authorship, individual contributions, originality, and structured navigation throughout the report.

❖ **Chapter One → Introduction**

√ Establishes the context and rationale for the project. It includes the background (overview of bug bounty platforms, the proposed organization's structure, mission, vision, core values, and business processes), statement of the problem (highlighting cybersecurity gaps in Ethiopia), objectives (general and specific), methodology (requirement gathering techniques such as interviews, questionnaires, document analysis of platforms like Bugcrowd and HackerOne,

literature review, and justification of the Object-Oriented approach), feasibility analysis (economic, technical, and time), beneficiaries and significance of the project, limitations, scope (detailing subsystems and boundaries), and this organization summary.

❖ **Chapter Two →System Features**

√ Presents a detailed analysis and object-oriented modeling of the proposed system.

It begins with a description of the existing informal and ad-hoc vulnerability

disclosure practices in Ethiopia. The chapter then provides comprehensive requirement analysis, including functional and non-functional requirements, business rules, UML diagrams (use case, activity, sequence, state chart, and class diagrams), user interface prototypes, CRC card analysis, conceptual class models, and identification of potential change cases for future enhancements.

- √ The document ends with References, citing all sources used (including OWASP guidelines, ISO/IEC 29147, academic papers, and local stakeholder inputs), and Appendices, containing supplementary materials such as sample interview/questionnaire forms, detailed wireframes, additional UML diagrams, and prototypes developed during the analysis phase.
- √ This RAD serves as a complete blueprint for the Crowdsourced Bug Bounty Platform, fulfilling the academic requirements while laying a solid foundation for future full-scale implementation, commercialization, and expansion across Ethiopia and Africa.

Chapter Two

System features

2.1 The Existing System

The Crowdsourced Bug Bounty Platform is a startup project currently in the planning and design phase, with no operational platform yet established. Therefore, there is no existing automated or semi-automated system within the startup organization itself. All activities related to vulnerability discovery and responsible disclosure that the future platform will manage are currently handled in a completely manual manner by ethical hackers and target organizations in Ethiopia.

The existing system refers to the prevailing practices in the Ethiopian cybersecurity ecosystem, where ethical hackers and organizations interact directly without any intermediary platform. These interactions are fully manual, unstructured, and depend entirely on personal initiative and communication channels.

2.1.1 How the Existing System Functions

The current process is 100% in person process, relying on email, phone calls, social media, or personal networks. There is no centralized portal, standardized forms, tracking system, or reward mechanism in place locally. Ethical hackers must individually identify and contact organizations, while organizations handle incoming reports on an ad-hoc basis with limited resources.The existing system operates as follows:

√ Organizations formally request security testing services from INSA or licensed private cybersecurity firms.

- √ Security assessments are conducted on-site or through controlled engagements, often requiring physical presence, official letters, and long approval processes.
- √ Testing is usually performed periodically, not continuously.
- √ Reports are delivered as static documents (PDF or printed reports).
- √ Vulnerability remediation is handled internally by the organization after receiving the report.
- √ There is limited or no structured follow-up, real-time communication, or verification of fixes.
- √ This approach is semi-automated and heavily dependent on institutional procedures.

2.2 Proposed System

The proposed Crowdsourced Bug Bounty Platform is a secure, scalable, and fully automated web-based system designed as a commercial startup solution to directly address the problems identified in the existing manual vulnerability disclosure practices in Ethiopia. It serves as a trusted, localized intermediary that connects Ethiopian organizations with ethical hackers, streamlining the entire lifecycle of vulnerability discovery, reporting, validation, remediation, and rewarding.

The system is modeled after leading global platforms such as Bugcrowd (with its emphasis on managed triage and researcher matching) and HackerOne (with strong researcher reputation systems and flexible program options), but tailored to the Ethiopian context with affordable pricing, local payment compliance, and support for regional cybersecurity talent development.

Key Features and Solutions to Existing Problems

- The proposed system eliminates the inefficiencies, risks, and barriers of the current manual approach by introducing the following automated and structured solutions.

➢ **Centralized Platform Access**

√ Replaces scattered email and social media contacts with a single, professional web portal where organizations and researchers can register, log in, and interact securely.

➢ **Structured Program Management**

√ Organizations can easily create and manage public, private (invite-only), or non-rewarded Vulnerability Disclosure Programs (VDPs). They define clear scope, rules, reward tiers, and severity guidelines solving the problem of unclear or non-existent disclosure policies.

➢ **Standardized Report Submission**

√ Ethical hackers submit vulnerabilities using guided forms with required fields (title, description, steps to reproduce, impact, attachments). This replaces unstructured emails and ensures complete, high-quality reports.

➢ **Automated and Managed Triage**

√ Platform staff (or organization teams) use built-in tools to intake, deduplicate, validate, prioritize, and assign severity (using a VRT-like taxonomy). This eliminates manual forwarding, delays, and duplicate handling issues.

➢ **Secure Collaboration and Communication**

√ In-platform messaging, comments, and real-time notifications replace insecure email threads, ensuring confidential and timely exchanges between researchers, organizations, and triage teams.

➢ **Fair Reward and Reputation System**

√ Valid reports trigger bounty awards, payout tracking, and automatic reputation updates with leaderboards—addressing the critical lack of incentives and recognition that currently discourages local researchers.

➢ **Role-Based Dashboards and Analytics**

√ Customized views for researchers (submissions, earnings, rankings), organizations (program performance, vulnerability trends), and administrators (platform oversight) .

➢ **Secure and Compliant Operations**

√ Built-in security features (encryption, role-based access, audit logs) and integration points for local payment systems (e.g., Telebirr, CBE Birr, bank transfers) ensure safe transactions and regulatory alignment.

Overall Benefits of the Proposed System

√ By automating and standardizing the entire workflow, the platform directly resolves the core problems of the current manual system

- Slow response times → Fast, tracked processing.
- Risk of public disclosure → Safe, coordinated handling.
- Low participation → Attractive rewards and reputation building.
- Inefficiency and errors → Structured, auditable processes.
- Dependency on foreign platforms → Affordable, local alternative with data sovereignty.

2.3 Requirement Analysis

2.3.1 Functional requirement

- The functional requirements define the core services and operations that the proposed crowdsourced bug bounty platform must provide. Each requirement is uniquely identified and prioritized based on its importance to system operation.

Table 1. Functional Requirement

Requirement ID	Functional Requirement	priority
FREQ-01	The system shall allow users to register and log in with different roles: Researcher (Ethical Hacker), Organization (Program Owner), Platform Staff (Triage Specialist), Finance Officer, and Administrator.	High
FREQ-02	The system shall support email verification, password recovery, and multi-factor authentication for all users.	High
FREQ-03	Organizations shall be able to create, edit, publish, and archive bug bounty programs (public, private/invite-only, or Vulnerability Disclosure Programs - VDP).	High
FREQ-04	Organizations shall define program details including scope (in-scope/out-of-scope assets), rules, reward tiers, severity guidelines, and disclosure policy.	High
FREQ-05	Researchers shall be able to browse, search, and join eligible active programs.	High
FREQ-06	Researchers shall submit vulnerability reports using a structured form with fields for title, description, steps to reproduce, impact assessment, severity suggestion,	High

	and file attachments (screenshots, PoC, videos).	
FREQ-07	The system shall allow platform staff/triage specialists to intake, review, deduplicate, validate, prioritize, and update report status (New, Triaged, Valid, Invalid, Duplicate, Resolved).	High
FREQ-08	The system shall assign severity ratings to reports using a standardized taxonomy (Vulnerability Rating Taxonomy - VRT).	High
FREQ-09	The system shall provide secure in-platform messaging and comment threads for collaboration between researchers, organizations, and triage staff.	Medium
FREQ-10	Organizations/Finance Officers shall approve bounties, set reward amounts, and mark reports for payout processing.	High
FREQ-11	The system shall track and update researcher reputation scores, signal strength, and display public leaderboards/rankings.	High
FREQ-12	The system shall send real-time notifications (email and in-platform) for key events such as new reports, status changes, messages, and bounty awards.	High
FREQ-13	The system shall provide role-specific dashboards, Researcher (submissions, earnings, rankings), Organization (program performance, reports, trends), Staff (triage queue), and Admin (platform overview).	High
FREQ-14	Administrators shall manage users, roles, programs, and perform platform audits and configurations.	High
FREQ-15	The system shall view analytics reports for vulnerability trends, program effectiveness, and researcher performance.	High
FREQ-16	The system shall support basic researcher-to-program matching based on skills, past performance, and program requirements (conceptual foundation for future AI enhancement).	Medium

FREQ-17	The system shall maintain an activity log and audit trail for all critical actions (report submission, status changes, bounty awards).	High
FREQ-18	The system shall allow researchers to view and track the status of their submitted reports.	High
FREQ-19	The system shall enable organizations to view and manage all reports submitted to their programs.	High
FREQ-20	The system shall integrate payout tracking (pending, processed) with placeholders for future local payment gateway integration (e.g., Telebirr, bank APIs).	Medium
FREQ-21	The system shall securely store vulnerability reports and related attachments.	High
FREQ-22	The system shall implement the SSDLC based on OWASP top 10 vulnerablity.	High

2.3.2 System Use case

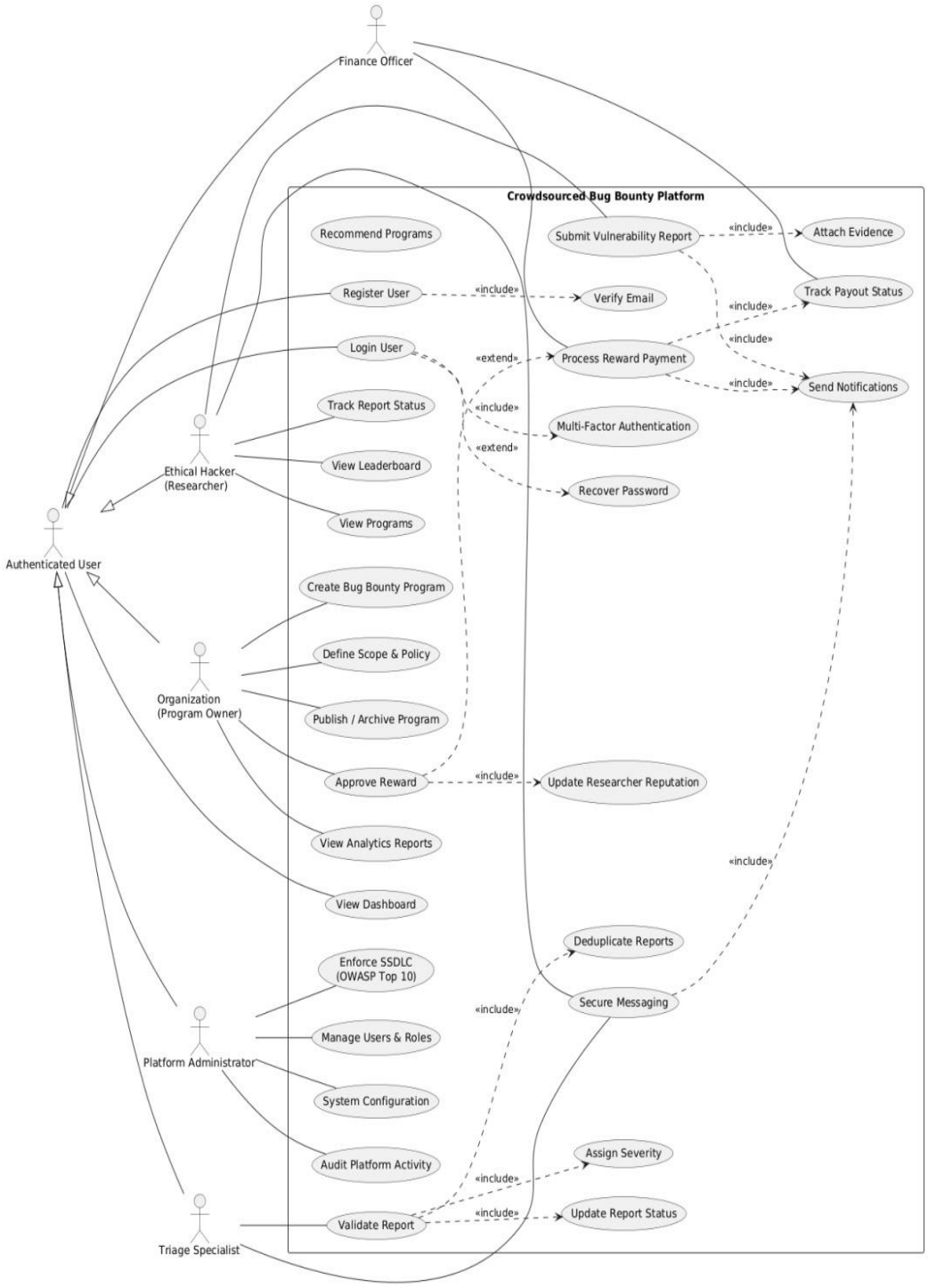
2.3.2.1 Use case Diagram

A use case diagram is a Unified Modeling Language (UML) diagram used to represent the functional behavior of a system and the way different actors interact with the system to achieve specific goals. It provides a high-level view of the system's functionality from the user's perspective.

In this project, the use case diagram illustrates the main functionalities of the Crowdsourced Bug Bounty Platform, the external actors involved (such as Ethical Hackers, Organizations, Platform Staff, and Administrators), and the interactions between these actors and the system.The diagram also demonstrates use case reusability and relationships using

√ **<<include>>** relationships to represent mandatory and reusable system behaviors,
√ **<<extend>>** relationships to show optional or conditional functionalities, and
√ **<<inherit>>** (generalization) relationships to model shared behaviors among different actors.

Image 1. Use case Diagram



2.3.2.2 Use Case Documentation

The following section documents the major use cases of the Crowdsourced Bug Bounty Platform. These use cases are derived from the functional requirements (FREQ-01 to FREQ-22) listed in Table 1 and the provided Use Case Diagram. Six primary use cases are detailed below to cover the core workflows of the system.

❖ UC-01→ Register User

√ Name : Register User

√ Actor : Unauthenticated User (prospective Researcher, Organization, Platform Staff, Finance Officer, Administrator)

√ Description : A new user creates an account on the platform by selecting a role and verifying their email.

√ Pre-condition : User has access to the platform homepage and a valid email address.

√ Post-condition : User account is created with the selected role, email is verified, and user is redirected to login or dashboard.

√ Priority : High (FREQ-01, FREQ-02)

- Basic Course of Action

  I. User navigates to the homepage and clicks Register (UI, Registration button).
  II. System displays registration form .
  III. System validates input and creates account.
  IV. System sends verification email (UI, Success message Verification email sent).
  V. User clicks verification link in email.
  VI. System verifies email and activates account (UI, Registration successful Login now).

- Alternate Course of Action

  I. A1. At step 3, Invalid input → System displays error messages.
  II. A2. At step 6, Verification link expired → User requests resend verification.

❖ UC-02→Submit Vulnerability Report

√ Name : Submit Vulnerability Report
√ Actor : Ethical Hacker (Researcher)
√ Description : Researcher submits a vulnerability report for an active program.
√ Pre-condition : Researcher is logged in and has joined an active program.
√ Post-condition : Report is saved, assigned a unique ID, and notification sent to triage team.
√ Priority : High (FREQ-05, FREQ-06, FREQ-12)

- Basic Course of Action

  I. Researcher navigates to program page and clicks Submit Report .
  II. System displays structured report form (UI, Fields – Title, Description, Steps to Reproduce, Impact, Suggested Severity, Attachments).
  III. Researcher fills all required fields and attaches evidence (supported formats are images, text, video).
  IV. Researcher submits the report.
  V. System validates completeness and saves report.
  VI. System sends notification to triage specialist and organization.

- Alternate Course of Action

I. A1. At step 3, Missing required fields → System highlights errors.
II. A2. At step 4,Potential duplicate detected → System warns and suggests viewing similar reports (includes Deduplicate Reports).

❖ UC-03→ Validate Report

- √ Name : Validate Report
- √ Actor : Triage Specialist
- √ Description : Triage specialist reviews, validates, and updates the status of a submitted report.
- √ Pre-condition : Triage Specialist is logged in, report exists in triage queue.
- √ Post-condition : Report status and severity updated, notifications sent.
- √ Priority : High (FREQ-07, FREQ-08, FREQ-12, FREQ-17).

- Basic Course of Action

  I. Triage Specialist opens triage dashboard (UI, Triage Queue list).
  II. Specialist selects a report to review.
  III. System displays full report details and attachments.
  IV. Specialist reproduces issue, assigns severity using VRT taxonomy, deduplicates if needed, and updates status (Business Rule BR-03, Severity must follow platform VRT guidelines).
  V. Specialist adds internal comments and saves changes.
  VI. System logs action in audit trail and sends notifications to researcher and organization.

- Alternate Course of Action

  I. A1. Report is duplicate → Status set to Duplicate and linked to original report.
  II. A2. Report invalid → Status set to Invalid with explanation.

❖ UC-04 → Create Bug Bounty Program

- √ Name : Create Bug Bounty Program
- √ Actor : Organization (Program Owner)
- √ Description : Organization creates and publishes a new bug bounty program.
- √ Pre-condition : Organization is logged in and has active subscription.
- √ Post-condition : Program is created, published, and visible to researchers.
- √ Priority : High (FREQ-03, FREQ-04)

- Basic Course of Action

I. Organization navigates to dashboard and clicks Create Program .
II. System displays program creation form (UI,Fields Name, Description, Type (Public/Private/VDP), Scope, Rules, Reward Tiers, Severity Guidelines).
III. Organization fills details and submits (Business Rule BR-04, In-scope assets must be explicitly listed).
IV. System saves draft.
V. Organization reviews and clicks Publish.
VI. System publishes program and notifies researchers/admin.

- Alternate Course of Action

I. A1. Incomplete scope/rules → System prevents publishing with error.

❖ UC-05 → Approve Reward

√ Name : Approve Reward
√ Actor : Organization / Finance Officer
√ Description : Approve bounty payment for a validated report.
√ Pre-condition : Report status is Valid, severity and reward tier defined.
√ Post-condition : Bounty approved, payout tracked, researcher reputation updated.
√ Priority : High (FREQ-10, FREQ-11, FREQ-20)

- Basic Course of Action

I. Actor opens report details (UI, Report view with Approve Reward button).
II. Actor reviews validation and sets/confirms reward amount (Business Rule BR-05, Reward must align with program tier).
III. Actor approves bounty.
IV. System updates report status, researcher reputation score, and payout status (Pending).
V. System sends notification to researcher.

- Alternate Course of Action

I. A1. Dispute over amount → Actor adjusts and adds comment.

❖ UC-06 → View Dashboard

√ Name : View Dashboard
√ Actor : All Authenticated Users (Researcher, Organization, Triage Specialist, Administrator)
√ Description : User views personalized dashboard with relevant metrics and activities.
√ Pre-condition : User is logged in.
√ Post-condition : Dashboard data displayed.
√ Priority : High (FREQ-13, FREQ-15)

- Basic Course of Action

I. User logs in.
II. System redirects to role-specific dashboard (UI, Customized views Researcher, submissions/earnings/leaderboard; Organization: program stats/reports).
III. User views charts, recent activities, and analytics.

- Alternate Course of Action

I. None significant.

2.3.3 Business Rule Documentation

Business rules define the policies, constraints, and operational guidelines that govern the Crowdsourced Bug Bounty Platform. Since the organization is a startup in the pre-launch/planning phase (no current operational system exists), these rules are proposed rules for the new commercial platform. They will be enforced automatically by the system (where possible) and manually by staff where required. The rules are derived from functional requirements (FREQ-01 to FREQ-22).

The following table lists the key business rules:

Table 2 .Bussiness Rule Documentation

Identifier	Name	Description	Reference	Revision History
BR-01	User Registration Eligibility	Users must be provide a valid email, and select one role (Researcher, Organization, Triage Specialist, Finance Officer, Administrator). Only one account per email is permitted.	FREQ-01, FREQ-02; Ethiopian Data Protection Proclamation No. 1321/2023	Version 1.0 (Initial) – 20/12/2025
BR-03	Severity Rating Assignment	Severity must be assigned using the platform's Vulnerability Rating Taxonomy (VRT) from Critical to Low. Triage specialists must provide justification in comments.	FREQ-08; VRT standard	Version 1.0 (Initial) – 20/12/2025
BR-04	Program Scope and Policy Definition	Organizations must explicitly define in-scope assets (e.g., domains, apps) and	FREQ-03, FREQ-04; ISO/IEC 29147	Version 1.0 (Initial) – 20/12/2025

		out-of-scope items. Any scope change must be re-approved and all active researchers notified.		
BR-05	Bounty Reward Determination	Organizations set custom bounty amounts based on report severity and their program policy.	FREQ-10, FREQ-20	Version 1.0 (Initial) – 20/12/2025
BR-06	Platform Commission (Triage Fee)	For managed triage programs, organizations pay the researcher bounty plus a 30% platform commissio n.	FREQ-23, FREQ-24	Version 1.0 (Initial) – 20/12/2025
BR-07	Duplicate Report Handling	First valid report receives full bounty (client-defined amount). Duplicate reports receive 50% of bounty if identical and submitted within 24 hours; otherwise, no reward. System auto-flags duplicates.	FREQ-07	Version 1.0 (Initial) – 20/12/2025
BR-08	Payout Processing Timeline	Approved bounties must be processed within 30 days.	FREQ-20; Ethiopian Revenue and	Version 1.0 (Initial) – 20/12/2025

		Delays require notification to researcher admins will notified	Customs Authority	
BR-09	Researcher Reputation Scoring	Reputation score starts at 0. Valid reports add 10–50 points (based on severity); invalid/duplicate reports deduct 20 points. Top 10 researchers displayed on public leaderboard.	FREQ-11; HackerOne signal system	Version 1.0 (Initial) – 20/12/2025
BR-10	Vulnerability Disclosure Timeline	Organizations must acknowledge reports within 24 hours and remediate within 90 days. Researchers agree not to disclose publicly until fixed or 90-day grace period expires.	FREQ-09; OWASP Responsible Disclosure Guidelines	Version 1.0 (Initial) – 20/12/2025

2.3.4 User Interface Prototype

The user interface prototypes for the Crowdsourced Bug Bounty Platform were developed using a prototype approach to gather and validate user interface requirements. Low-fidelity wireframes were created and reviewed with stakeholders (ethical hackers and organizations) during requirement gathering interviews and questionnaires. These prototypes directly align with the documented use cases (UC-01 to UC-06) and functional requirements (FREQ-01 to FREQ-22), ensuring intuitive user interaction, role-based views, and support for core workflows.The prototypes emphasizes

- ➢ Simplicity and Accessibility : Clean layouts suitable for low-bandwidth environments in Ethiopia.
- ➢ Responsive Design: Mobile-friendly using Bootstrap (as planned in 1.4.3).

Registration Page (Referenced in UC-01: Register User)

- √ Description : Full-screen form centered on the page.
- √ Key UI Elements: Role selection dropdown, input fields (Full Name, Email, Password, Confirm Password), "Register" button, link to Login page, password strength meter.
- √ Alignment : Supports FREQ-01 and FREQ-02 (email verification triggered after submission).

Login Page (Referenced in UC-01 and all use cases requiring authentication)

- √ Description : Minimalist form with platform logo at top.
- √ Key UI Elements: Email and Password fields, Login button, Forgot Password? link, MFA code input .
- √ Alignment : Role-based redirect to dashboards after login.

Researcher Dashboard (Referenced in UC-06 View Dashboard)

√ Description : Grid layout with cards and tables.

√ Key UI Elements : Welcome message, reputation score badge, earnings summary, leaderboard snippet, list of active programs, Browse Programs and Submit Report buttons, notifications sidebar (FREQ-12, FREQ-13, FREQ-18**).**

Submit Vulnerability Report Form (Referenced in UC-02)

√ Description : Step-by-step or single-page form with progress indicator.

√ Key UI Elements : Required fields (Title, Description textarea, Steps to Reproduce numbered list, Impact dropdown, Suggested Severity), file upload drag-and-drop area (preview thumbnails), Submit and Save Draft buttons (FREQ-06, FREQ-21).

Create Bug Bounty Program Form (Referenced in UC-04)

√ Description: Tabbed or multi-step wizard form.

√ Key UI Elements: Tabs for General Info, Scope (in-scope/out-of-scope text boxes), Rules textarea, Program Type radio buttons (Public/Private/VDP), Save Draft and Publish buttons (FREQ-03, FREQ-04)**.**

Triage Queue and Report Validation Page (Referenced in UC-03)

√ Description : Table-based queue with detail modal.

Report Detail and Reward Approval Page (Referenced in UC-05)

√ Description : Detailed view with approval section at bottom.

√ Key UI Elements : Report content display, triage comments, custom bounty amount input field, commission breakdown

Organization Dashboard (Referenced in UC-06)

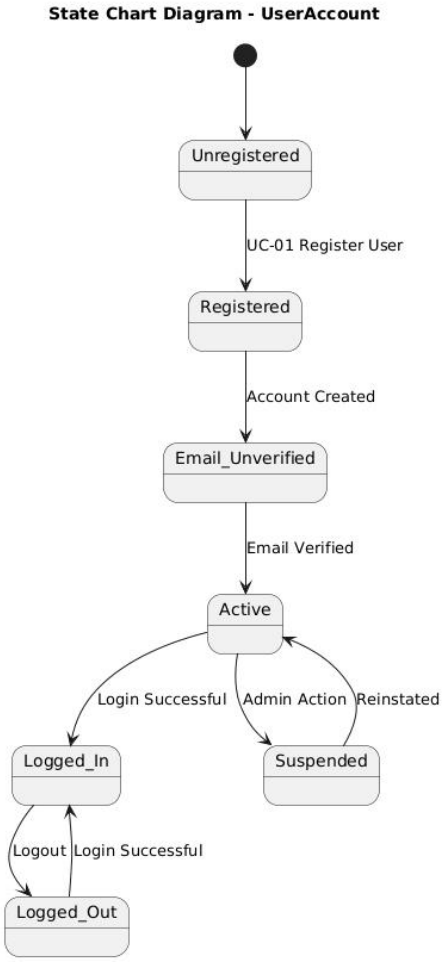
√ Description : Analytics-focused layout with charts.

√ Key UI Elements : Program list, vulnerability trend charts, reports table, financial summary card (bounties paid), quick links to create program or view reports (FREQ-13, FREQ-15, FREQ-19).

2.3.5 State chart diagram

State chart diagrams illustrate the lifecycle of key objects in the Crowdsourced Bug Bounty Platform, showing how instances of each class transition between states in response to events. The following diagrams are provided for the most important domain classes, based on the object-oriented analysis, use cases (UC-01 to UC-06), and functional requirements (FREQ-01 to FREQ-22).

Image 2. User Registration_State chart Diagram



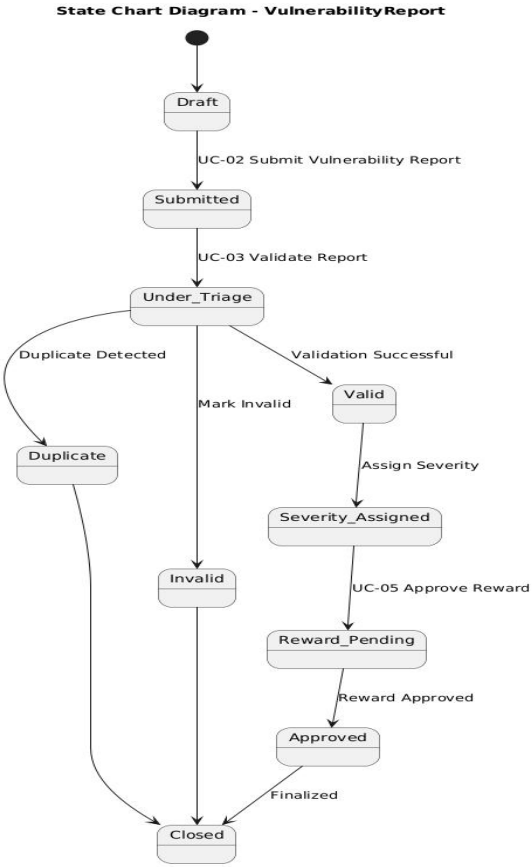
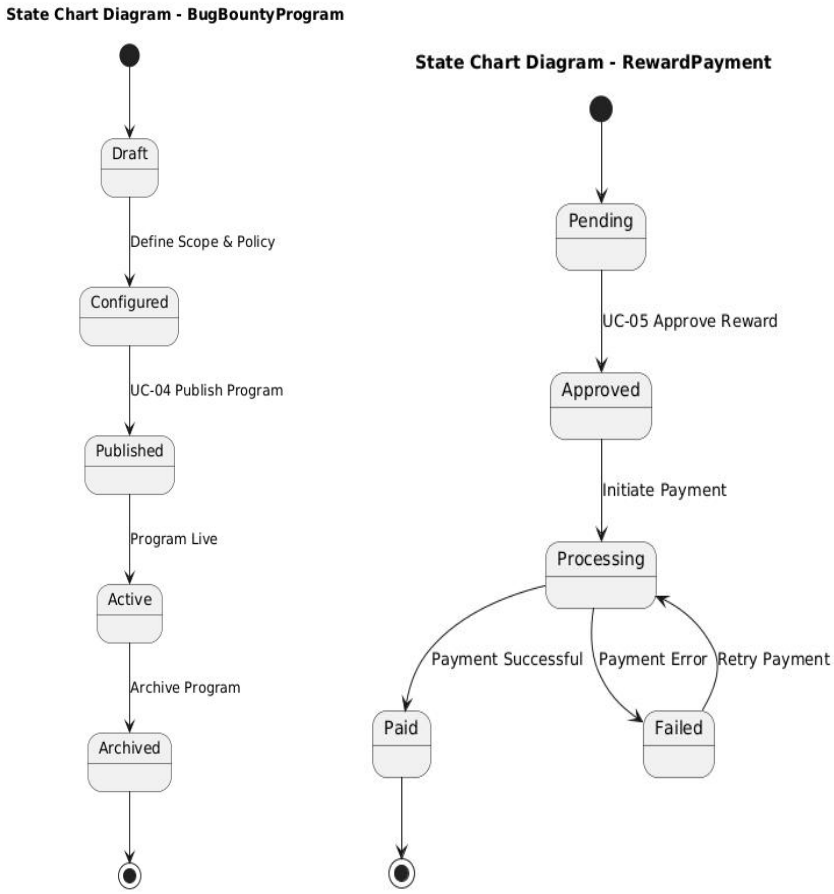


Image 3.vulnerablity report

Image 4.program creation



2.3.6 Activity Diagram

Activity diagrams model the detailed flow of activities and decisions within the major use cases of the Crowdsourced Bug Bounty Platform. The following diagrams are provided for the six primary use cases (UC-01 to UC-06) documented in 2.3.2.2, showing the sequence of actions, decisions, and parallel processes performed by actors and the system.

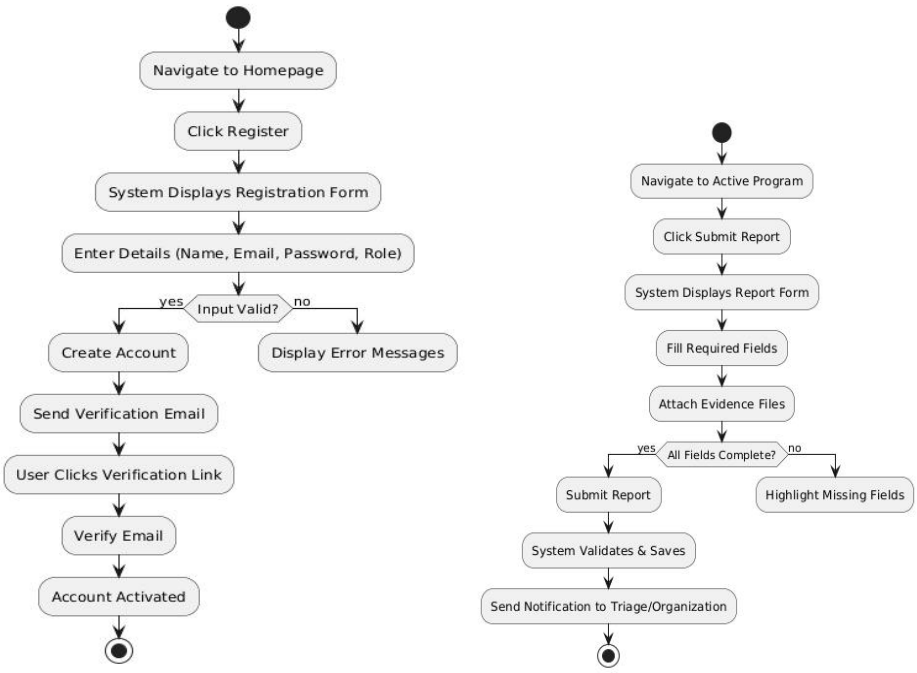
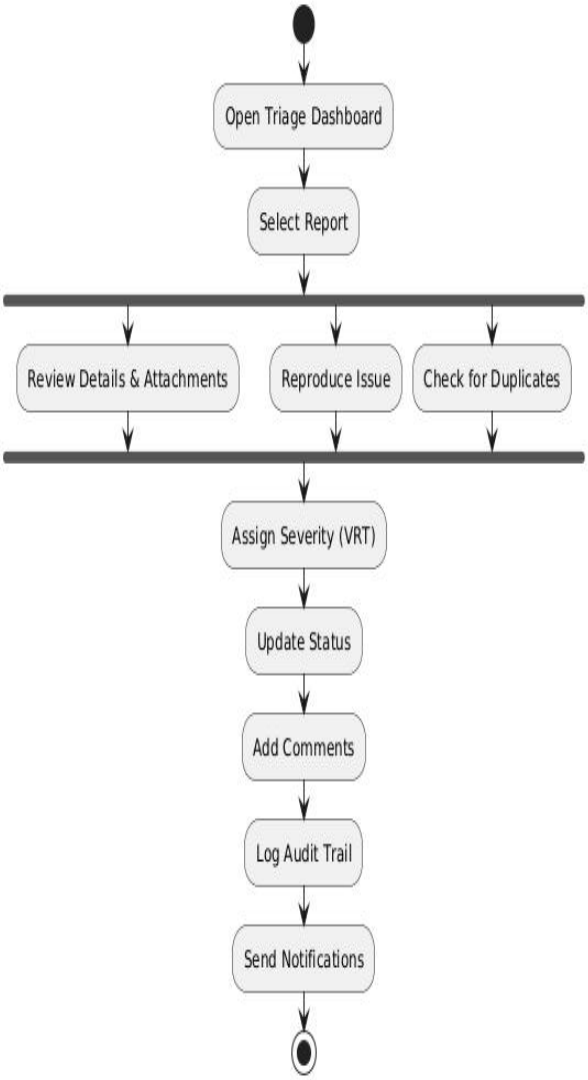


Image 5. activity diagram user registration

Image 6.validate report



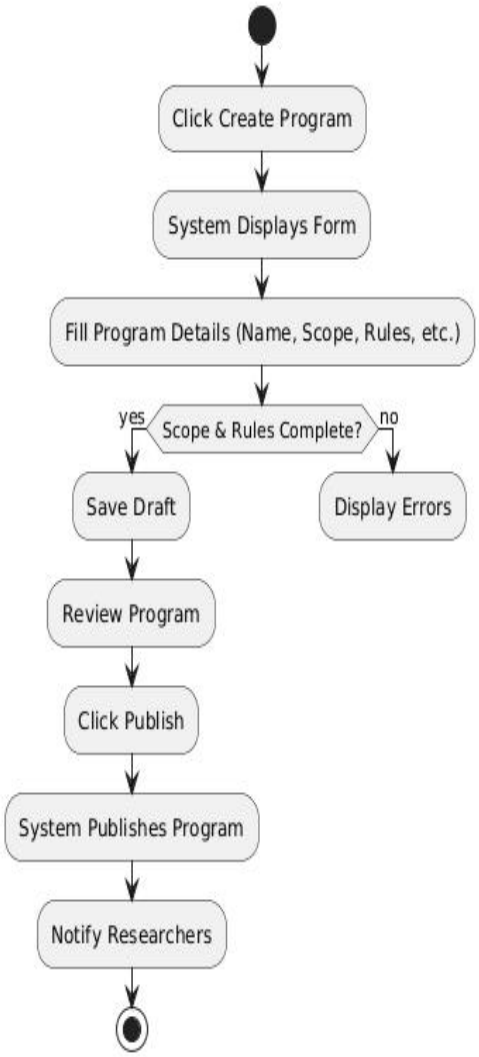


Image 7. program creation_activity

Image 8. Reward Approval

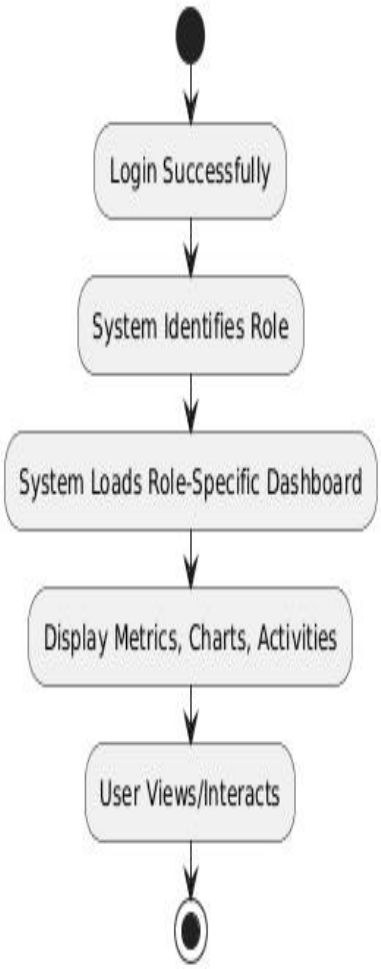


Image 9.Dashbord

2.3.7 Sequence Diagram

Sequence diagrams model the chronological interactions between objects (and actors) to achieve the functionality of each use case. The following diagrams are provided for the six primary use cases (UC-01 to UC-06), showing message flows, object lifecycles, and system responses.

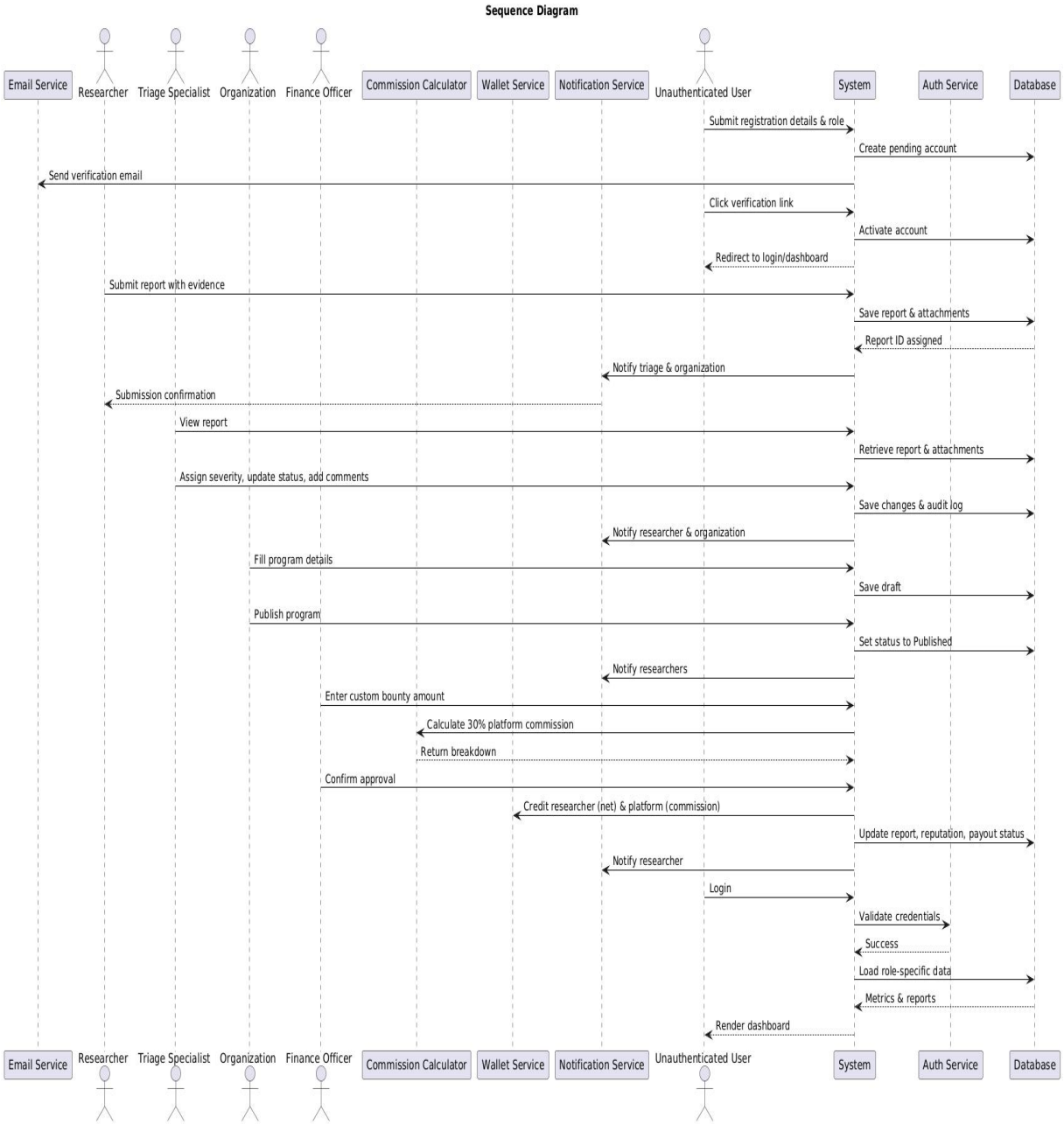


Image 10.Sequence Diagram

2.3.8 Analysis Class Model

The analysis class model represents the static structure of the Crowdsourced Bug Bounty Platform using an object-oriented approach. It identifies key domain classes, their attributes, operations (methods), relationships, multiplicity, and roles, derived from the use cases, functional requirements (FREQ-01 to FREQ-22), activity diagrams, state charts, and sequence diagrams.

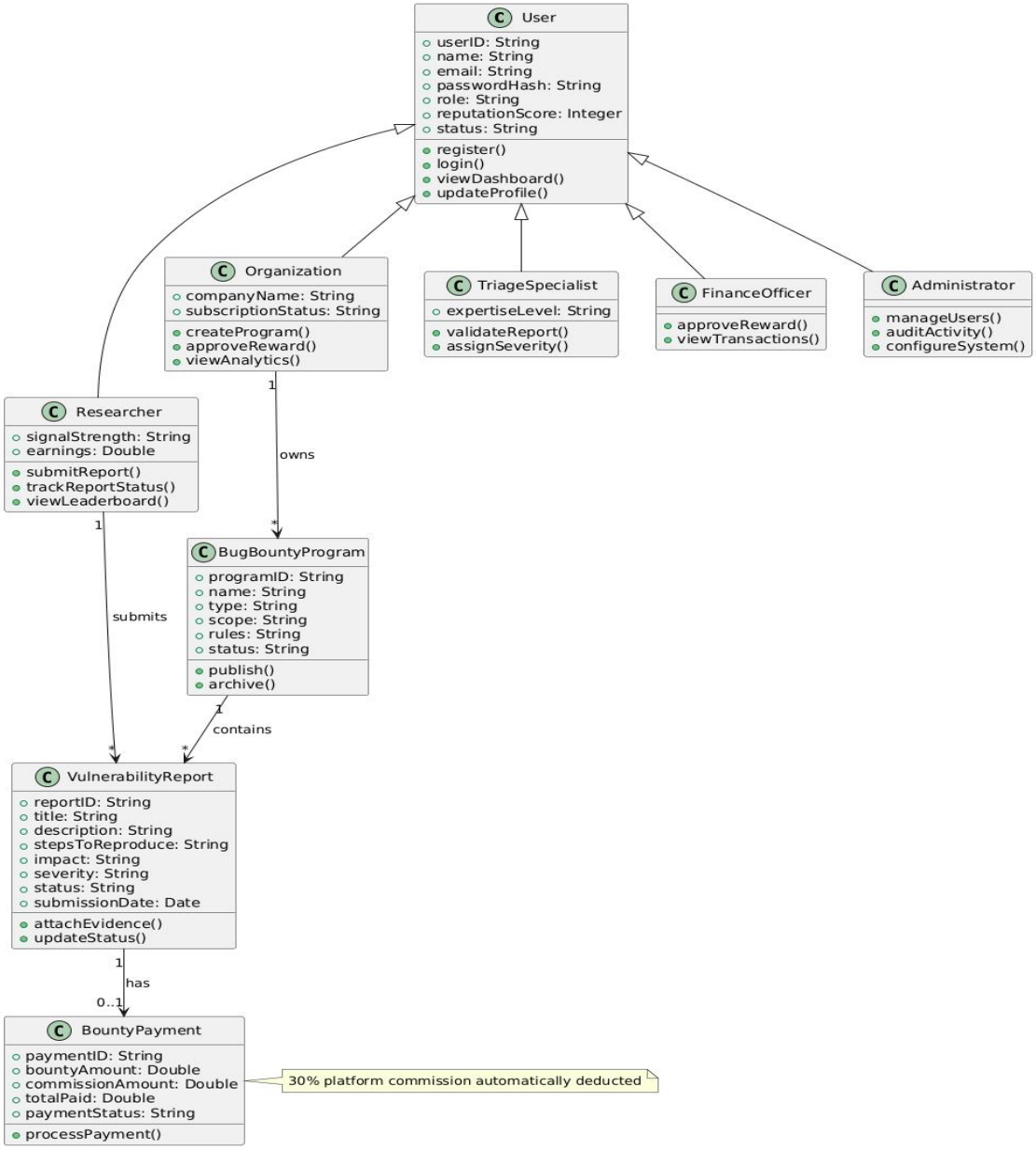


Image 11. Analysis Class Model

2.3.9 Logic Model

The logic model section presents algorithms/pseudo-code for the critical processes of the Bug Bounty Platform. These pseudo-codes describe the step-by-step logic for key operations derived from the use cases (UC-01 to UC-06), functional requirements (FREQ-01 to FREQ-22), and previous modeling diagrams. They provide a clear blueprint for implementation.

❖ Algorithm for UC-01 : User Registration

```
BEGIN RegisterUser(userDetails, selectedRole)
  IF userDetails.email EXISTS IN Database THEN
    RETURN "Email already registered"
  END IF
  IF userDetails.password NOT meet complexity requirements THEN
    RETURN "Password too weak"
  END IF
  CREATE pendingUser WITH userDetails AND selectedRole
  SET pendingUser.status = "PendingVerification"
  SAVE pendingUser TO Database
  verificationLink = GENERATE unique verification token
  SEND email TO userDetails.email WITH verificationLink
  RETURN "Verification email sent"
END
BEGIN VerifyEmail(token)
```

```
        FIND user WITH matching token
        IF user FOUND AND token valid THEN
            SET user.status = "Active"
            REMOVE token
            SAVE user
            RETURN "Account activated successfully"
        ELSE
            RETURN "Invalid or expired link"
        END IF
    END
```

❖ Algorithm for UC-02 : Submit Vulnerability Report

```
BEGIN Submit Vulnerability Report(researcherID,programID, reportDetails,
attachments)
    IF researcher NOT joined programID THEN
        RETURN "Not eligible for this program"
    END IF
    IF reportDetails missing required fields THEN
        RETURN "Incomplete report"
    END IF
    IF attachments size > 10MB OR count > 5 THEN
        RETURN "Attachment limits exceeded"
    END IF
    CREATE newReport
    SET newReport.ID = GENERATE unique ID
    SET newReport.status = "New"
    SET newReport.submissionDate = CURRENT timestamp
    ASSIGN reportDetails and attachments TO newReport
    SAVE newReport TO Database
    SEND notification TO triage specialists AND program owner
```

```
    RETURN "Report submitted successfully - ID: " + newReport.ID
  END
```

❖ Algorithm for UC-03 : Validate Report

```
  BEGIN ValidateReport(triageID, reportID, severity, status, comments)
    LOAD report FROM Database USING reportID
    IF report.status != "New" AND report.status != "Triaged" THEN
      RETURN "Report already processed"
    END IF
    // Deduplication check
similar Reports = SEARCH reports WITH similar title/description IN same program
    IF similarReports FOUND THEN
      SET report.status = "Duplicate"
      LINK report TO original similarReport
    ELSE
      SET report.severity = severity (using VRT taxonomy)
      SET report.status = status  // Valid, Invalid, or Triaged
    END IF
    ADD comments TO report
    LOG audit trail (triageID, action, timestamp)
    SAVE report
    SEND notification TO researcher AND organization
    RETURN "Report updated successfully"
  END
```

- Algorithm for UC-04 : Create Bug Bounty Program

```
BEGIN CreateBugBountyProgram(orgID, programDetails)
    IF programDetails.scope OR rules missing THEN
        RETURN "Scope and rules are mandatory"
    END IF
    CREATE newProgram
    SET newProgram.ID = GENERATE unique ID
    SET newProgram.status = "Draft"
    ASSIGN programDetails TO newProgram
    SAVE newProgram TO Database
    RETURN "Draft saved - ID: " + newProgram.ID
BEGIN PublishProgram(programID)
    LOAD program FROM Database
    IF program.status != "Draft" THEN
        RETURN "Only drafts can be published"
    END IF
    SET program.status = "Published"
    SET program.publishDate = CURRENT date
    SAVE program
    SEND notification TO all researchers (or eligible based on matching)
    RETURN "Program published successfully"
END
```

- Algorithm for UC-05 : Approve Reward (with 30% Commission)

```
BEGIN ApproveReward(financeID, reportID, researcherBountyAmount)
    LOAD report FROM Database
    IF report.status != "Valid" THEN
      RETURN "Only valid reports can receive bounty"
    END IF
    IF researcherBountyAmount < 10 THEN
      RETURN "Minimum bounty is $10"
    END IF
    commissionAmount = researcherBountyAmount * 0.30
    totalAmount = researcherBountyAmount + commissionAmount
    CREATE paymentRecord
    SET paymentRecord.researcherAmount = researcherBountyAmount
    SET paymentRecord.commissionAmount = commissionAmount
    SET paymentRecord.totalAmount = totalAmount
    SET paymentRecord.status = "Approved"
    CREDIT researcher wallet WITH researcherBountyAmount
    CREDIT platform wallet WITH commissionAmount
    UPDATE report.status = "Resolved"
    UPDATE researcher.reputationScore = researcher.reputationScore +
  severityBasedPoints
    LOG transaction in audit trail
    SEND notification TO researcher ("Bounty of $" + researcherBountyAmount + "
  approved")
    RETURN "Reward approved - Total charged: $" + totalAmount
  END
```

- Algorithm for UC-06 : View Dashboard

```
BEGIN ViewDashboard(userID)
  LOAD user FROM Database
  DETERMINE user.role
```

```
    SWITCH user.role
        CASE "Researcher":
            LOAD submittedReports, earnings, reputationRank, activePrograms
        CASE "Organization":
            LOAD ownedPrograms, submittedReportsCount, vulnerabilityTrends
        CASE "TriageSpecialist":
            LOAD triageQueue (New/Triaged reports)
        CASE "Administrator":
            LOAD platformStats (totalUsers, totalReports, revenue)
        DEFAULT:
            LOAD general overview
    END SWITCH
    RENDER dashboard WITH loaded data and charts
    RETURN dashboard view
END
```

2.4 Non functional requirement

Non-functional requirements specify the quality attributes and technical constraints of the Crowdsourced Bug Bounty Platform. These requirements ensure the system is secure, performant, reliable, usable, scalable, and maintainable, especially given its commercial nature and handling of sensitive vulnerability data.

Table 3. Non Functional Requirement

ID	Category	Requirement	Justification / Standard
NFR-01	Security	The system shall use HTTPS (TLS 1.3) for all communications and	Prevents man-in-the-middle attacks and ensures data confidentiality.

		enforce secure headers (HSTS, CSP, X-Frame-Options).	
NFR-02	Security	The system shall implement role-based access control (RBAC) with least-privilege principle for all roles (Researcher, Organization, Triage Specialist, Finance Officer, Administrator).	Ensures users can only access authorized functions and data (FREQ-01, FREQ-14).
NFR-03	Security	The system shall hash passwords using bcrypt or Argon2 and enforce strong password policies (minimum 12 characters, mixed case, numbers, symbols).	Protects against credential stuffing and brute-force attacks.
NFR-04	Security	The system shall sanitize all inputs and use prepared statements to prevent SQL injection, XSS, and CSRF attacks.	Follows OWASP Top 10 (FREQ-22).
NFR-05	Security	The system shall encrypt sensitive data at rest (e.g., vulnerability details, attachments) using AES-256.	Protects stored vulnerability information from unauthorized access.
NFR-06	Security	The system shall implement rate limiting and CAPTCHA on login/report submission to prevent brute-force and spam.	Mitigates automated abuse.
NFR-07	Security	The system shall log all critical actions (registration, report submission, status changes, bounty approvals) with timestamps and user IDs for audit trails.	Supports compliance and forensic analysis (FREQ-17).
NFR-08	Performance	The system shall respond to user requests within 2 seconds under	Ensures good user experience even on low-bandwidth connections in Ethiopia.

		normal load (95th percentile).	
NFR-09	Performance	The system shall support at least 1,000,00 concurrent users with no more than 5% degradation in response time.	Scalability for growing researcher and organization base.
NFR-10	Reliability	The system shall achieve above 90 % uptime annually, excluding scheduled maintenance.	Ensures continuous availability for time-sensitive vulnerability reporting.
NFR-11	Reliability	The system shall implement automatic backups daily and retain data for at least 90 days.	Enables disaster recovery and compliance with responsible disclosure timelines.
NFR-12	Usability	The system shall be intuitive, requiring no more than 3 clicks to perform core actions (e.g., submit report, view dashboard).	Reduces learning curve for non-technical users and researchers.
NFR-13	Usability	The system shall be fully responsive and usable on mobile devices (screen width ≥320px).	Supports access from smartphones in Ethiopia with high mobile penetration.
NFR-14	Usability	The system shall support English as the primary language, with clear, simple terminology.	Ensures accessibility for local and international users.
NFR-15	Scalability	The system shall be designed using a microservices/cloud-native architecture to allow horizontal scaling.	Prepares for future growth across Africa.
NFR-16	Maintainability	The system shall follow clean code practices, be well-documented, and use modular design.	Facilitates future enhancements and commercialization.
NFR-17	Compatibility	The system shall support modern browsers (latest Chrome, Firefox, Safari, Edge) and degrade gracefully on older versions.	Ensures broad accessibility.

NFR-18	Legal/Compliance	The system shall comply with Ethiopian data protection laws (Proclamation No. 1321/2023) and international responsible disclosure standards (ISO/IEC 29147).	Ensures legal operation and ethical handling of vulnerability data.

2.5 System Requirement

2.5.1 Hardware requirements

The Bug Bounty Platform is a web-based commercial application designed to be deployed on cloud infrastructure (e.g., AWS, DigitalOcean, or local Ethiopian data centers) for scalability and cost-effectiveness. The following hardware specifications represent the minimum and recommended requirements for successful deployment and operation.

Table 4. Hardware Requirement

Component	Minimum Requirement	Recommended Requirement	Justification
CPU	4 vCPUs (2.5 GHz or higher)	8 vCPUs (3.0 GHz or higher, e.g., Intel Xeon or AMD EPYC)	Handles concurrent report submissions, triage, notifications, and commission calculations.
RAM	16 GB	32 GB or higher	Supports in-memory caching (Redis), database queries, and file processing (attachments).
Storage (SSD)	100 GB	500 GB or higher (NVMe SSD preferred)	Stores vulnerability reports, attachments (PoC files, screenshots), database, and logs.
Network	5 Mbs bandwidth	10 Gbps bandwidth with load balancer	Ensures fast response times and secure file uploads/downloads.
Other	RAID 1 or	Auto-scaling instance	Provides reliability and disaster

	higher for redundancy	group with backup	recovery.

2.5.2 Software requirements

The Bug Bounty Platform is a web-based application designed to run on modern software environments. The following specifications outline the required operating systems, browsers, servers, databases, frameworks, and other software products (with versions) needed for successful deployment, development, and user access

Table 5. Software Requirement

Component	Required Software / Version	Justification
Operating System	Ubuntu 22.04 LTS (or later) or equivalent Linux distribution (e.g., CentOS Stream 9)	Stable, secure, widely supported for web servers and cloud deployment.
Web Server	Nginx 1.18 or higher	High performance, reverse proxy, load balancing, and HTTPS termination.
Application Runtime	Python 3.10 or higher (with Django 4.2+) or Node.js 18+ (with Express/NestJS)	Backend framework for robust, secure web application (as per 1.4.3 methodology).
Database Management System	PostgreSQL 14 or higher	Reliable, ACID-compliant, supports complex queries for reports and analytics.
Caching	Redis 7.0 or higher	Fast in-memory caching for sessions, notifications, and rate limiting.
Background Tasks	Celery 5.3+ with Redis/RabbitMQ as broker	Asynchronous processing (email notifications, commission calculations).
Containerization	Docker 24.0+ and Docker Compose	Consistent deployment across development, testing, and production.
Cloud/Orchestration	Kubernetes (optional for scaling) or Docker Swarm	Enables auto-scaling and high availability in production.

2.6 Key Abstraction with CRC Analysis

CRC (Class-Responsibility-Collaboration) cards are used to identify the key domain objects in the Crowdsourced Bug Bounty Platform, their responsibilities, and how they collaborate. This analysis evolves into the class diagram and ensures a robust object-oriented design.

Table 6 . CRC Analysis

Class	Responsibilities	Collaborations
User	Register, login, view dashboard, update profile	System, Database, Email Service
Researcher	Submit vulnerability report, attach evidence, track report status, view leaderboard	VulnerabilityReport, BugBountyProgram, Notification Service
Organization	Create/publish bug bounty program, define scope/rules, approve reward, view analytics	BugBountyProgram, VulnerabilityReport, FinanceOfficer
TriageSpecialist	Validate report, assign severity, deduplicate, update status, add comments	VulnerabilityReport, Database, Notification Service
FinanceOfficer	Approve custom bounty amount, confirm commission payment	VulnerabilityReport, Commission Calculator, Wallet Service
Administrator	Manage users/roles, audit activity, configure system	All classes, Database
BugBountyProgram	Store program details	Organization, VulnerabilityReport

	(scope, rules, type), change status (Draft → Published → Archived)	
VulnerabilityReport	Store report details (title, description, evidence), change status (New → Valid → Resolved)	Researcher, TriageSpecialist, BugBountyProgram, BountyPayment
BountyPayment	Calculate commission (30%), credit wallets, record transaction	FinanceOfficer, Wallet Service, Database

2.6.1 Conceptual Modeling: Class Diagram

The conceptual class diagram serves as the building block for the Crowdsourced Bug Bounty Platform. It represents the core domain objects, their attributes, operations, relationships, and multiplicity, refined from the CRC analysis (2.6) and the detailed analysis class model (2.3.8).

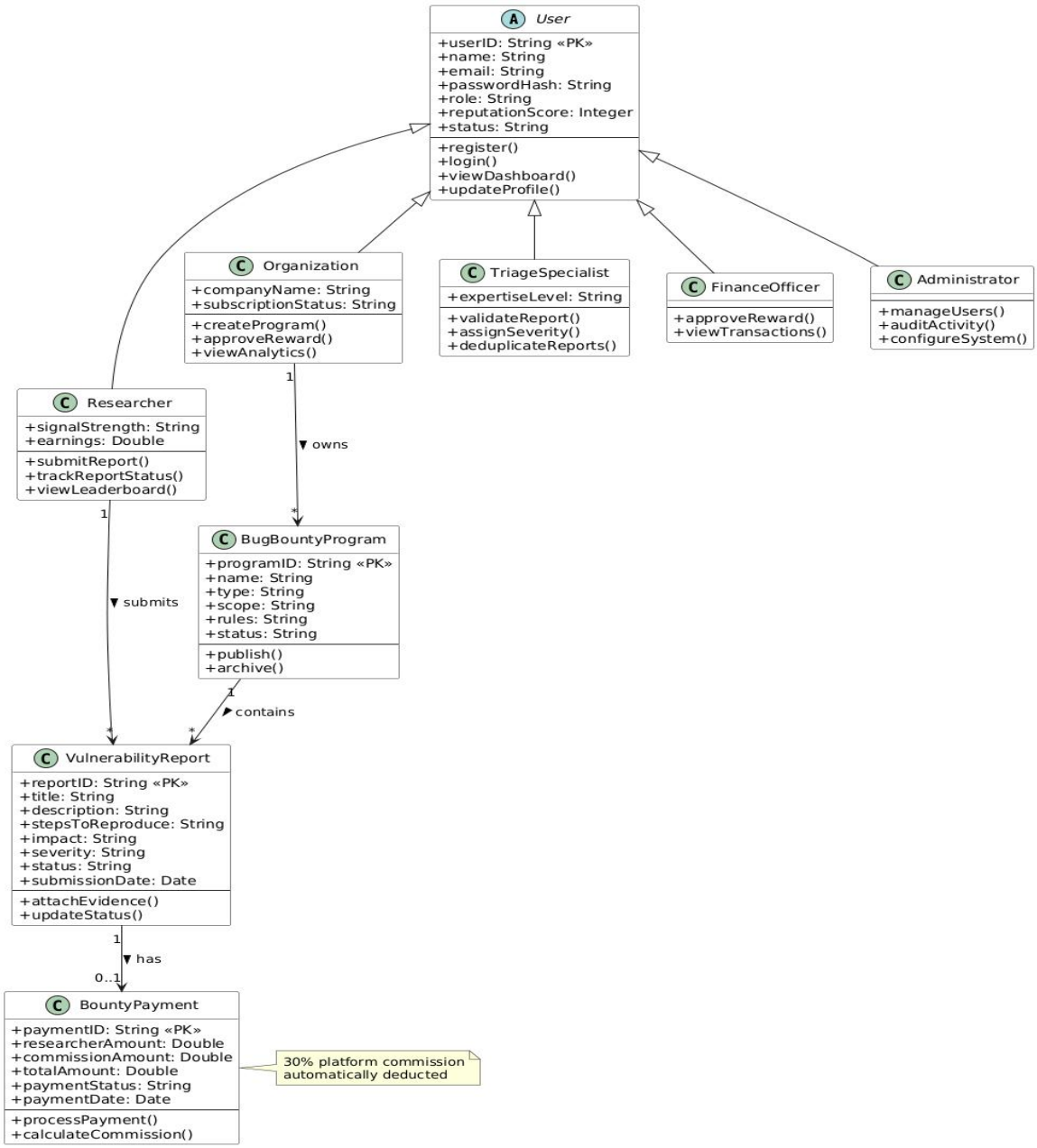


Image 12. Class Diagram

The diagram illustrates

- **Inheritance :** User as an abstract superclass generalized to specialized roles (Researcher, Organization, TriageSpecialist, FinanceOfficer, Administrator).
- **Associations and Multiplicity**

  - One Organization owns many BugBountyProgram (1..*).
  - One Researcher submits many VulnerabilityReport (1..*).
  - One BugBountyProgram contains many VulnerabilityReport (0..*).
  - One VulnerabilityReport may have one BountyPayment (0..1).

√ Automatic calculation of 30% platform commission in BountyPayment.

√ This conceptual model provides the structural foundation for implementation, ensuring modularity, scalability, and clear object interactions.

2.6.2 Identifying change cases

Change cases anticipate future modifications to the system requirements. The following potential changes are identified with likelihood and impact.

Table 7. Change Cases

Change Case ID	Description	Likelihood	Impact	Affected Components
CC-01	Add support for cryptocurrency payouts (e.g., Bitcoin, Ethereum)	Medium	High – Requires new wallet integration, tax compliance, security audit	BountyPayment, Wallet Service, FREQ-20
CC-02	Introduce AI-powered researcher-program matching and automated	High	High – Improves efficiency but needs training data and validation	TriageSpecialist, Researcher, FREQ-16

	triage			
CC-03	Add mobile applications (iOS and Android) for researchers and organizations	High	Medium – Expands accessibility, requires native development	All UI components, responsive design
CC-04	Support multilingual interface (e.g., Amharic, French)	Medium	Medium – Enhances regional adoption in Africa	UI Prototypes, NFR-14
CC-05	Integrate with external tools (Jira, Slack, GitHub) for report syncing	Medium	Medium – Improves enterprise adoption	Notification Service, VulnerabilityReport
CC-06	Implement Penetration Testing as a Service (PTaaS) as premium add-on	Medium	High – Major new revenue stream and functionality	New subsystem, pricing model

2.6.3 User Interface Prototyping

User interface prototypes were developed to visualize the Crowdsourced Bug Bounty Platform and gather additional requirements from stakeholders during interviews and questionnaires. Low-fidelity wireframes were created using Figma and Draw.io to demonstrate key screens and user flows. These prototypes helped validate usability, role-based access, and interaction patterns aligned with the use cases (UC-01 to UC-06) and functional requirements (FREQ-01 to FREQ-22).

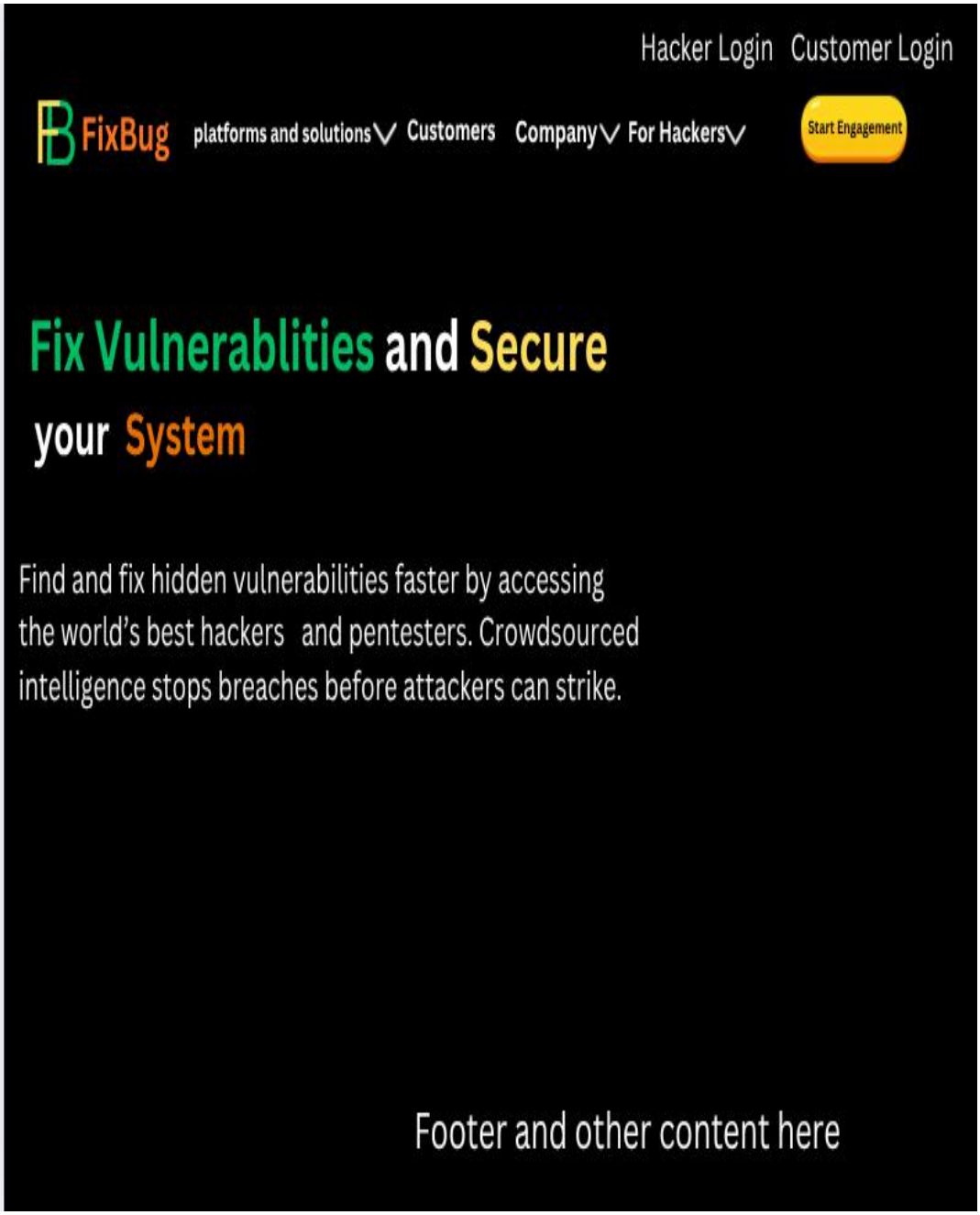
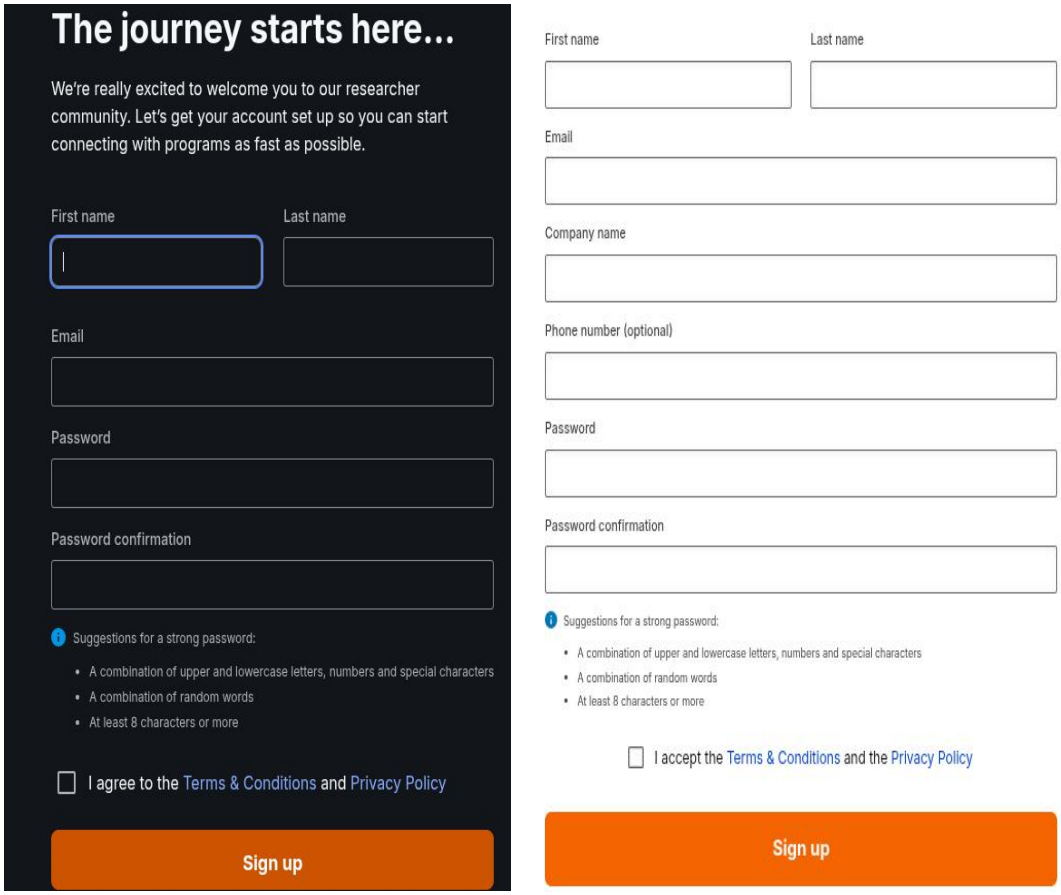


Image 13. Home page prototype

Image 14.user registration prototype



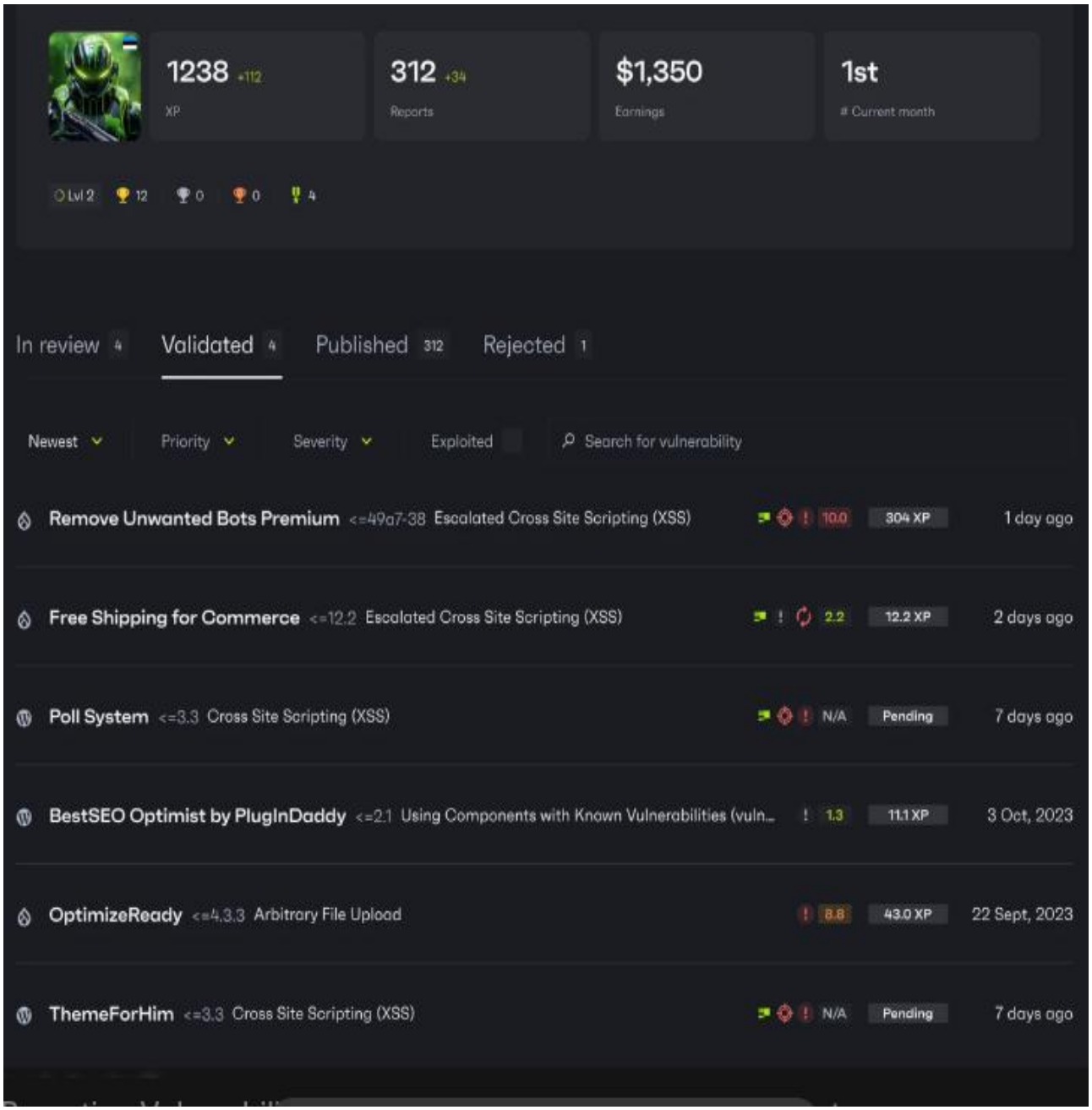


Image 15.Researcher Dashbord

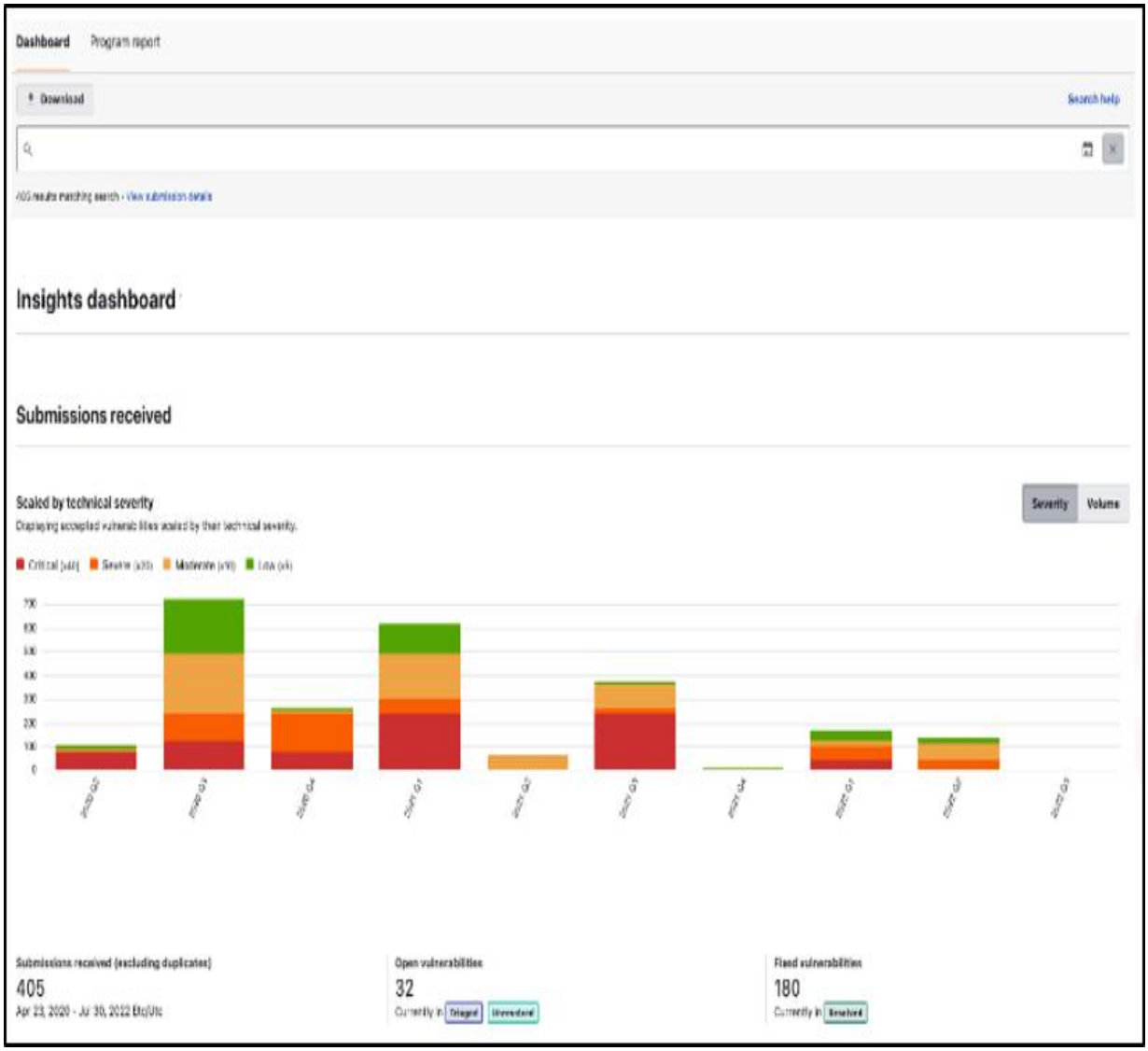


Image 16.less featured Analytics

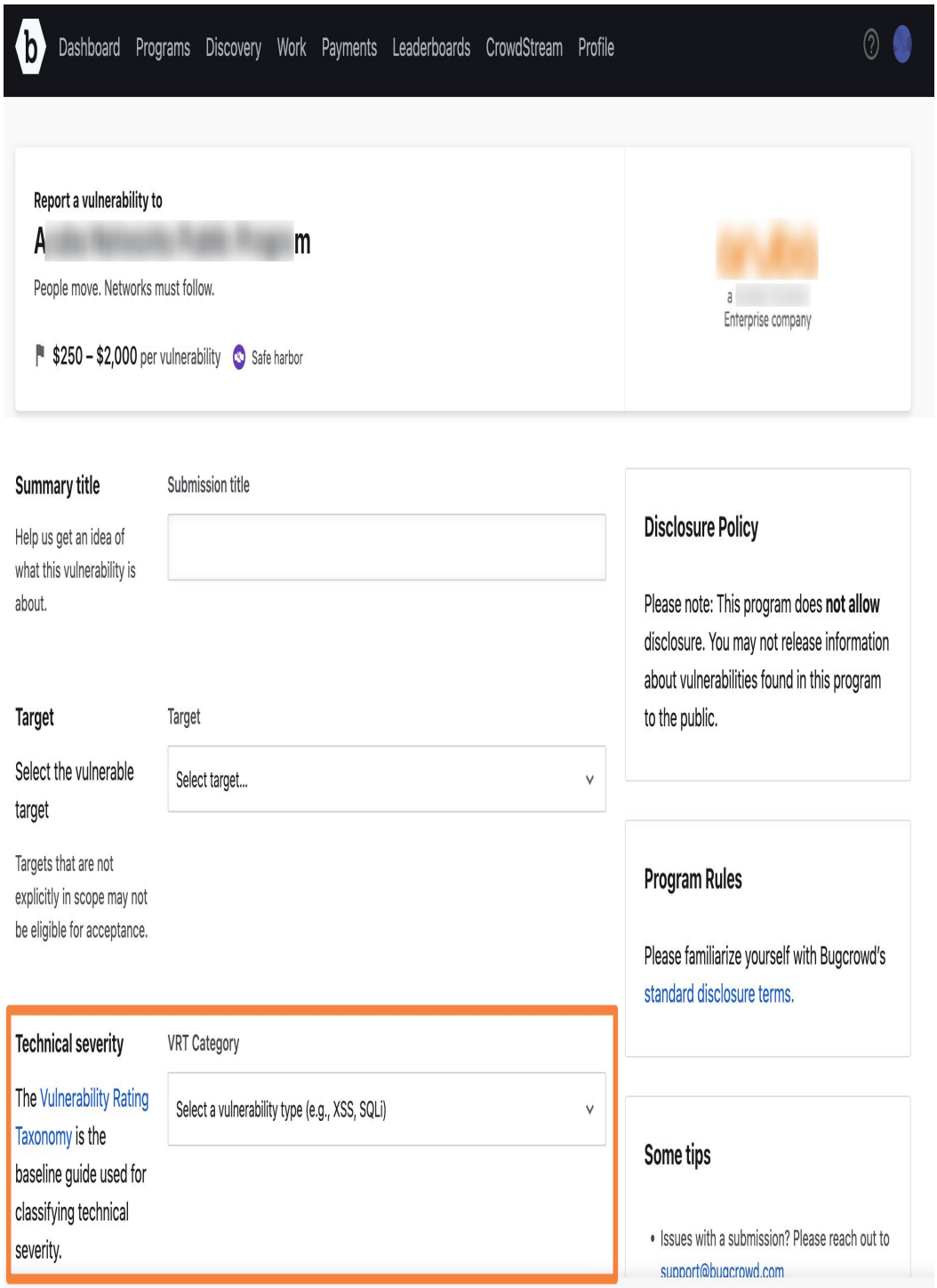


Image 17. Report Submission prototype

Dashboard Programs Discovery Work Payments Leaderboards CrowdStream Profile

Attachments (optional)

Attach proof-of-concept scripts, screenshots, screen recordings, etc.

Add attachments

You can attach up to 20 files. Please keep individual upload size under 400MiB.

You can embed attachments (.jpg/.gif/.png, smaller than 50MB) into the Markdown fields. You can copy the embed code using the 'Copy as Markdown' button.

Collaborate (optional)

Add researchers and collaborate on this submission.

Add by username and define the reward split between each researcher.

You

India

100 %

Add collaborator

Confirmation

Confirm your submission adheres to:

- the brief
- and Bugcrowd's terms & conditions

I have followed the **brief** and agree to **Bugcrowd's terms & conditions**

Report vulnerability

Save draft Delete Saved a few seconds ago

Image 18.report submission 2

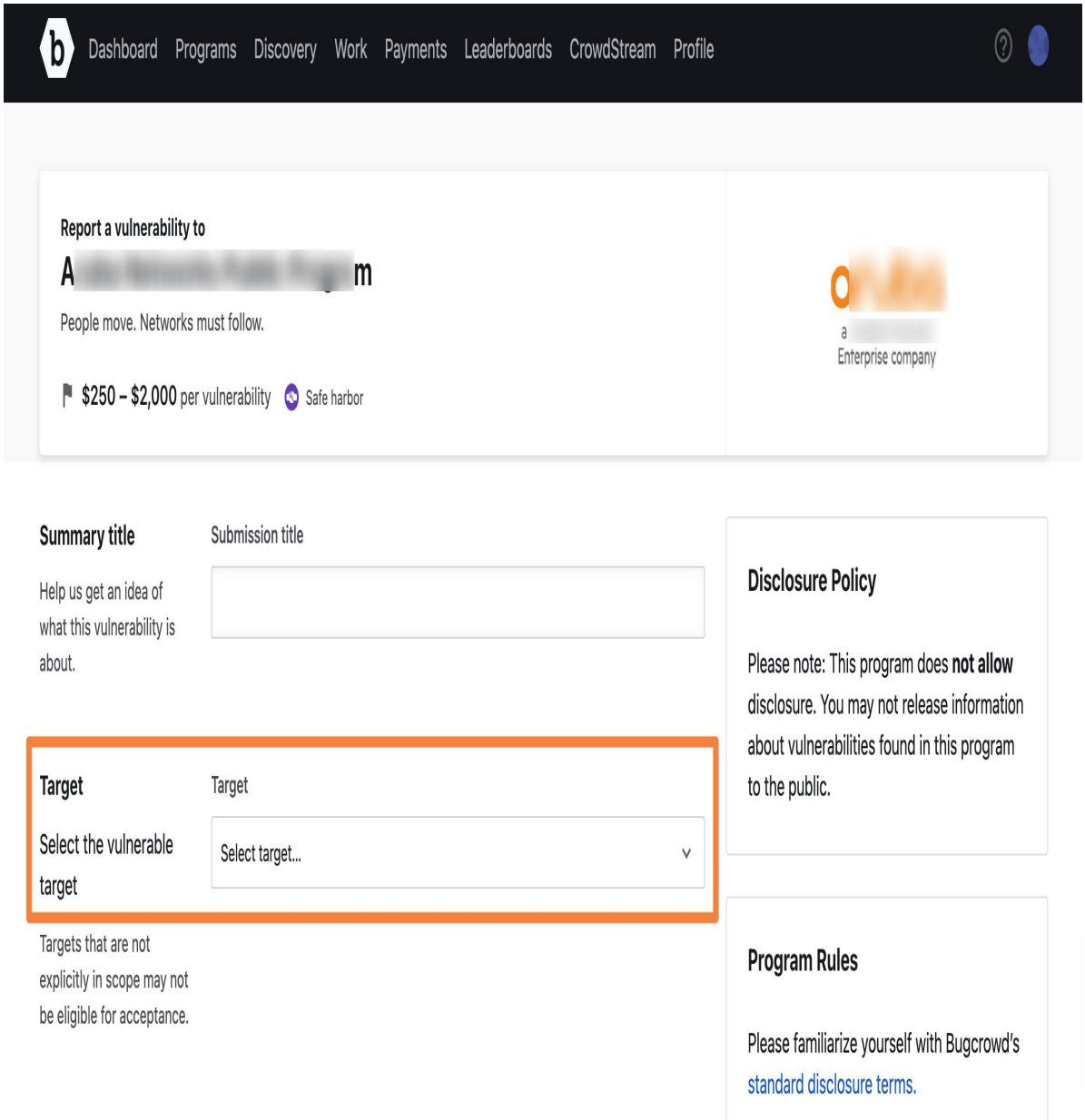


Image 19. Bounty program Creation prototype

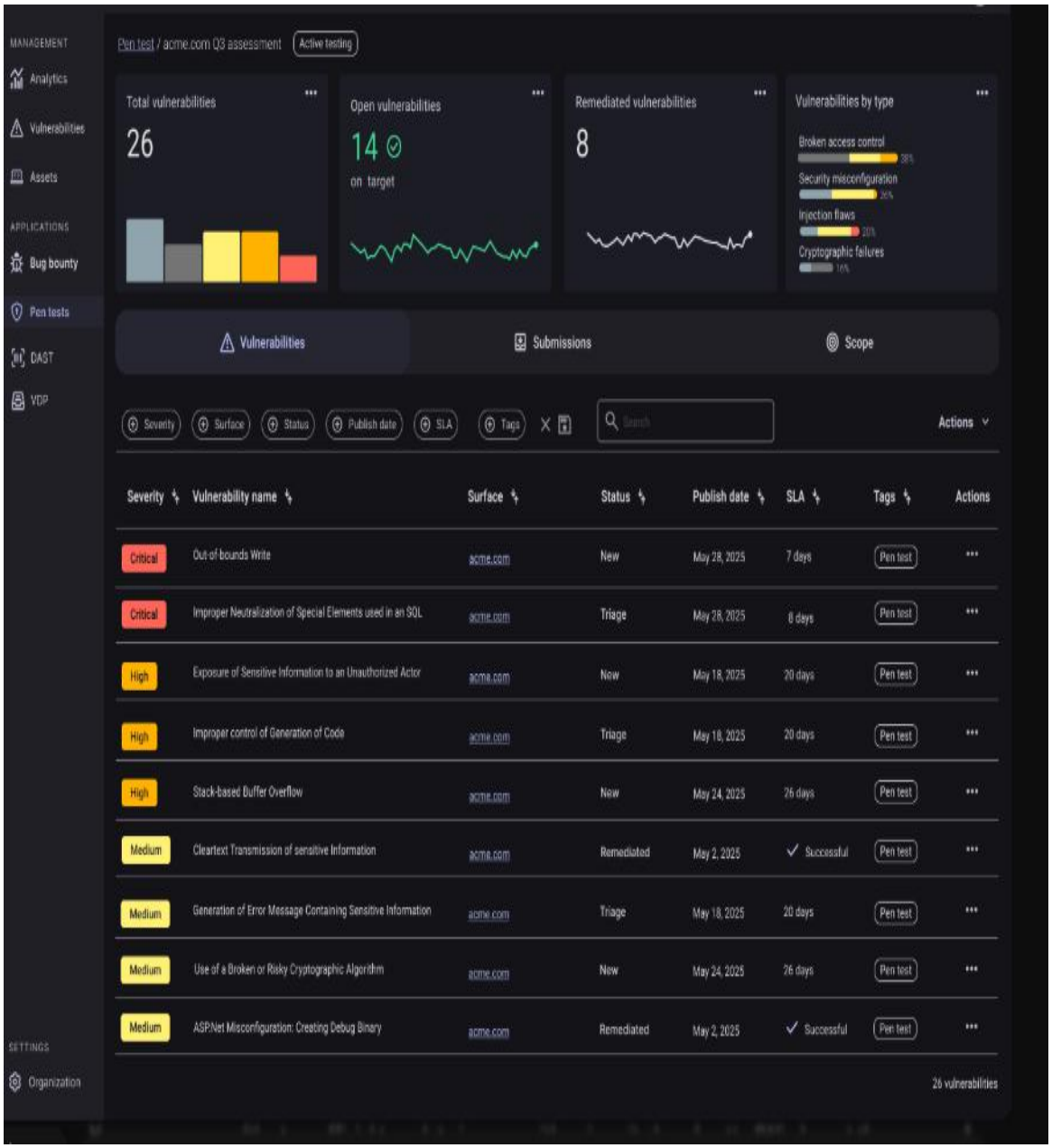


Image 20. Customer pentest Dashboard

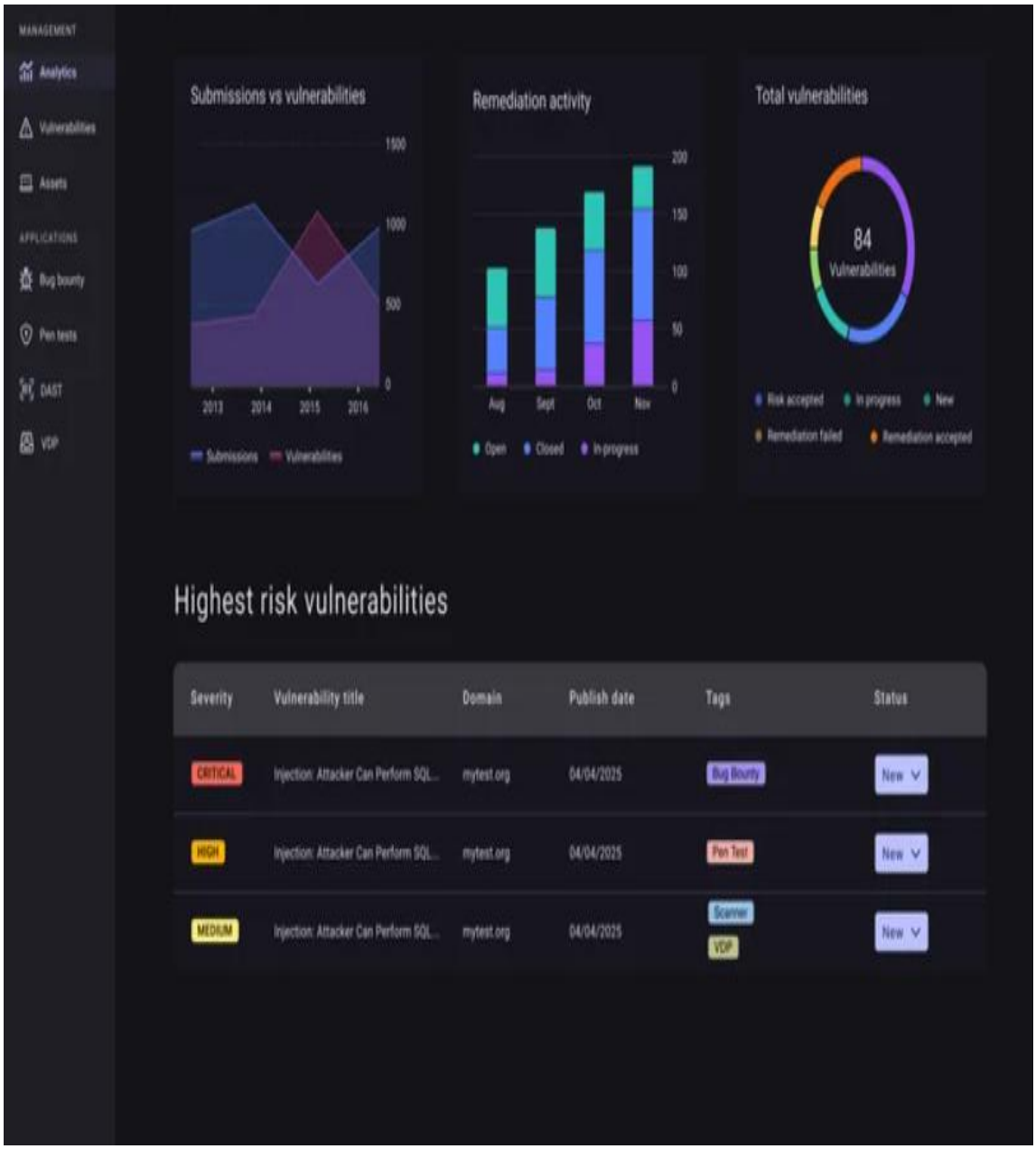


Image 21. Customer Analytics Dashboard

References

- √ Bugcrowd. (2025). *Bugcrowd Platform Documentation and Vulnerability Rating Taxonomy (VRT)*. Retrieved from https://docs.bugcrowd.com/
- √ HackerOne. (2025). *HackerOne Platform Help Center and Signal Reputation System*. Retrieved from https://docs.hackerone.com/
- √ OWASP Foundation. (2023). *OWASP Top 10 Web Application Security Risks*. Retrieved from https://owasp.org/www-project-top-ten/
- √ OWASP Foundation. (2023). *Responsible Disclosure Guidelines*. Retrieved from https://owasp.org/www-policy/legal/responsible-disclosure
- √ International Organization for Standardization (ISO). (2018). *ISO/IEC 29147:2018 - Information technology — Security techniques — Vulnerability disclosure*. Geneva: ISO.
- √ National Bank of Ethiopia. (2024). *Guidelines on Electronic Payments and Tax Withholding*. Retrieved from https://nbe.gov.et/
- √ Ethiopian Revenue and Customs Authority. (2024). *Income Tax Proclamation for Digital Services*. Retrieved from https://erca.gov.et/
- √ Government of Ethiopia. (2023). *Data Protection Proclamation No. 1321/2023*. Federal Negarit Gazeta.
- √ Bahir Dar University, Faculty of Computing. (2025). *Annex I: Documentation Template for Requirement Analysis Document (RAD)*. Internal guideline document.
- √ Unified Modeling Language (UML). (2024). *OMG UML Specification Version 2.5.1*. Object Management Group. Retrieved from https://www.omg.org/spec/UML/

Appendeces

Sample Questionnaire Used During Requirement Gathering

Questionnaire for Ethical Hackers (Researchers)

- Have you ever participated in bug bounty programs?
- What features do you consider most important in a bug bounty platform?
- What challenges do you face when reporting vulnerabilities to Ethiopian organizations currently?
- Would you participate in a local Ethiopian bug bounty platform? Why or why not?
- Any suggestions for features in a new Ethiopian bug bounty platform?

Questionnaire for Organizations

- Does your organization have a responsible disclosure policy?
- How do you currently handle vulnerability reports?
- What is your budget range for security testing (penetration testing or bug bounty)?
- Would you launch a bug bounty program if a trusted local platform existed?
- What concerns do you have about crowdsourced security testing (e.g., data confidentiality, false positives)?
- Preferred payout currency and method for researchers?)

Interview Questions Used During Requirement Gathering

Interview Guide for Security Specialists

- Describe your current process for receiving and triaging vulnerability reports.
- What tools do you use for vulnerability management?
- How long does it typically take to acknowledge and validate a report?
- What severity classification system do you use (e.g., CVSS, custom)?
- Have you used international bug bounty platforms? What was your experience?